

Shared Autonomy: Designing User Oversight for
LLM-Based Smartphone Agents

MASTER'S THESIS

written by

Andreas Schurawlev

Matriculation number: 11138619

submitted to

TECHNISCHE HOCHSCHULE KÖLN
CAMPUS GUMMERSBACH
FACULTY OF COMPUTER SCIENCE AND
ENGINEERING SCIENCES

in the study programme

MEDIA INFORMATICS (M.SC.)

First examiner: Prof. Dr. Matthias Böhmer
Technische Hochschule Köln

Second examiner: David Ferdinand Petersen
Technische Hochschule Köln

Gummersbach, September 2026

Addresses: Andreas Schurawlev
andreas.schurawlev@smail.th-koeln.de

Prof. Dr. Matthias Böhmer
Technische Hochschule Köln
Campus Gummersbach
Cologne Institute for Digital Ecosystems
Steinmüllerallee 1
51643 Gummersbach
matthias.boehmer@th-koeln.de

David Ferdinand Petersen
Technische Hochschule Köln
Campus Gummersbach
Cologne Institute for Digital Ecosystems
Steinmüllerallee 1
51643 Gummersbach
david.petersen@th-koeln.de

Abstract

Large language model (LLM)-based smartphone agents can carry out tasks across multiple applications, but their actions are not always predictable. Users therefore need enough insight and control to notice and correct errors without having to supervise every action. This thesis presents Caddie, a smartphone agent that makes its current action visible and allows users to confirm, pause, correct, or abort execution. In a controlled study, 18 participants each completed six cross-application tasks in total, two under each of three oversight conditions: confirmation of each task-advancing step, confirmation only of the final step, and no mandatory confirmation. Voluntary intervention remained available in all conditions, which used the same planned task sequences and controlled error variants. The study examined interaction effort, subjective workload, perceived control, and the perceived ability to intervene in time. It also compared participant-reported error detection across low- and high-criticality tasks.

Participants made a median of seven confirmations per task in the stepwise-confirmation condition, one in the final-checkpoint condition, and none in the condition without mandatory confirmation. The planned contrasts did not establish workload differences or the predicted ordering of perceived control across all three conditions. With a final checkpoint, participants reported a greater ability to intervene in time than when no confirmation was required, although support depended on the analysis method. The analysis did not establish higher reported error detection with more frequent mandatory confirmation. In the primary analysis, participants were more likely to report detecting errors in the selected high-criticality tasks than in the low-criticality tasks, although some sensitivity analyses did not support this difference. Most participants preferred no mandatory confirmation for low-consequence tasks and stepwise confirmation for high-consequence tasks. These paired preferences suggest that the preferred oversight level depends on task consequences rather than one confirmation policy applied uniformly.

Contents

Abstract	i
List of Figures	iv
List of Tables	v
Glossary	vi
1 Introduction	1
1.1 Motivation and Problem	1
1.2 Research Gap and Objective	1
1.3 Research Questions	2
1.4 Approach and Contributions	2
1.5 Thesis Structure	3
2 Background and Related Work	4
2.1 From Large Language Models to Interactive Agents	4
2.2 LLM-Based Agents for Mobile Graphical User Interfaces	5
2.3 Human Control, Automation, and Shared Autonomy	6
2.4 Oversight, Intervention, and Error Detection	6
2.5 Closest Systems and Empirical Studies	8
2.6 Synthesis and Research Positioning	14
3 Caddie: System Design and Implementation	15
3.1 Design Goals and Requirements	15
3.2 Architecture Boundary and Execution Modes	16
3.3 Normal-Mode Agent, Tools, and Context	18
3.3.1 Agent Loop and Model Gateway	18
3.3.2 Built-in Tools and MCP Extensions	20
3.3.3 Skills, Retrieval-Augmented Generation, and Replay Guidance	20
3.4 Android Observation and Verified Action Execution	22
3.5 Oversight and Intervention	24
3.6 Controlled Study Runtime and Instrumentation	28
3.7 Development Evidence and Technical Validation	30
3.7.1 Model and Retrieval Evaluations During Development	31
3.7.2 Execution Strategies and Task Efficiency	32
3.7.3 Observation, Execution, and Study Apparatus	33
3.8 Reproducing Caddie and Technical Boundaries	34
4 Study Design	38
4.1 Research Questions and Hypotheses	38
4.2 Experimental Design	39
4.2.1 Oversight Conditions	40
4.2.2 Task Criticality and Controlled Errors	40
4.2.3 Within-Participant Allocation and Counterbalancing	41

Contents

4.3	Apparatus and Study Materials	42
4.3.1	Caddie Study Implementation	42
4.3.2	Tasks and Controlled Study Applications	43
4.3.3	Questionnaires and Interview	43
4.4	Participants and Data Handling	45
4.5	Procedure	46
4.6	Measures and Operationalization	47
4.6.1	RQ1: Effort, Workload, and Control	48
4.6.2	RQ2: Participant-Reported Detection of Introduced Errors	48
4.6.3	Secondary and Exploratory Outcomes	49
4.7	Analysis Plan	50
4.7.1	Interaction Effort, Workload, and Control	52
4.7.2	Reported Error Detection and Its Timing	53
4.8	Study Replication	55
5	Results	57
5.1	Participant Flow and Data Quality	57
5.2	Task-Criticality Manipulation Check	57
5.3	RQ1: Interaction Effort, Workload, and Control	58
5.3.1	Required Decisions and Response Time	58
5.3.2	Subjective Workload	59
5.3.3	Perceived Control and Timely Intervention	59
5.4	RQ2: Participant-Reported Detection of Introduced Errors	61
5.4.1	Detection by Task Criticality	61
5.4.2	Detection by Oversight Condition	62
5.4.3	Criticality Difference Across Oversight Conditions	63
5.4.4	Reported Detection Stage	64
5.4.5	Reported Anomalies Without an Assigned Error	64
5.5	Secondary and Exploratory Outcomes	65
5.5.1	Perceived Result Match and Assessment Confidence	65
5.5.2	Trial Duration	66
5.5.3	Recorded Explicit Corrections	66
5.5.4	Confirmation Frequency and Attention Cues	67
5.5.5	Understanding, Predictability, and Trust	67
5.5.6	Overall and Consequence-Specific Oversight Preferences	68
5.5.7	Interview Themes	69
6	Discussion	70
6.1	Effects of Required Oversight	70
6.2	Participant-Reported Detection of Introduced Errors Across Tasks	71
6.3	Design Implications for Shared Autonomy	72
6.4	Limitations	73
7	Conclusion and Future Work	75
7.1	Conclusion	75
7.2	Future Work	76
	Bibliography	78

List of Figures

2.1	User involvement in a mobile GUI agent loop	7
2.2	AgentLens visual modalities	10
3.1	Phone-owned Caddie runtime	18
3.2	Caddie application icon and setup states	19
3.3	Normal-mode iteration and dispatch routes	22
3.4	Accessibility snapshot and target resolution	23
3.5	Task start and participant-facing oversight states	25
3.6	Normal-mode connection and language settings	35
3.7	Phone-hosted study portal and investigator view	36
4.1	Within-participant allocation and 18-slot matrix	42
4.2	Reconstructed high-criticality fixture outcomes	44
4.3	German participant interface	47
4.4	Conducted study sequence	47
5.1	Participant-level workload and control ratings	60
5.2	Model-estimated detection probabilities	63
5.3	Control-context ratings by oversight condition	67
5.4	Consequence-specific oversight preference transitions	68

List of Tables

2.1	Closest systems and empirical studies	13
3.1	Banking trial specification	29
3.2	Caddie development and validation evidence	31
3.3	Observable and fast execution comparison	33
3.4	Accessibility and UI Automator observation comparison	33
4.1	Task pairs, controlled alternatives, and commit boundaries	41
5.1	Consequence ratings by task criticality	58
5.2	Required participant decisions and response times	59
5.3	Raw NASA-TLX workload scores by oversight condition	59
5.4	Perceived control and timely intervention	60
5.5	Detection of introduced errors by task criticality	61
5.6	Detection of introduced errors by oversight condition	62
5.7	Reported stage of error detection	64
5.8	Perceived result match and confidence by assigned-error status	65
5.9	Task duration by oversight condition	66
5.10	Recorded corrections in assigned-error trials	66
5.11	Understanding, predictability, and trust ratings	68
5.12	Themes in written interview responses	69

Glossary

These entries summarize how the terms are used in this thesis.

Accessibility service. An Android service that can receive interface events, inspect accessible interface elements, and perform supported actions [3].

Android Debug Bridge (ADB). A command-line tool for communicating with an Android device from a development computer [2].

Commit step / commit boundary. The step at which the requested task change takes effect. The commit boundary separates preparation from this step.

Confidence interval (CI). Here, an interval from a method designed to include the true parameter in 95% of repeated samples under the model assumptions [15].

Confirmation / checkpoint. A required decision point at which the user approves or rejects continuation of the agent’s execution.

Context. The input available to the model when generating a response, including instructions, conversation history, and tool results.

Controlled error. A predefined incorrect task value deliberately inserted into an assigned study trial.

Counterbalancing. Distributing conditions and task orders across participants to reduce their confounding with order. It does not guarantee complete balance.

Credible interval. Here, an interval containing 95% of the posterior probability, conditional on the observed data, model, and prior assumptions [15].

Effective sample size (ESS). An estimate of how many independent draws would provide comparable precision to dependent posterior draws. It is not the number of participants [34].

Embedding. A numerical representation of text used to compare a query with stored passages for retrieval [22].

Holm adjustment. A procedure that adjusts p -values across a family of comparisons to control the probability of at least one false rejection [18].

Indeterminate detection classification. A study classification used when the evidence cannot establish error exposure or clearly distinguish detection from non-detection.

Inference. Using a trained model to produce an output from the supplied input, rather than training its parameters.

Interquartile range (IQR). The difference between the 75th and 25th percentiles. Bracketed IQR summaries in this thesis report the two quartiles themselves.

Intervention. A user action that changes the course of execution, such as rejecting an action, stopping execution, or providing a correction.

Large language model (LLM). A neural language model trained on large text collections. The generative models discussed here predict successive tokens from context [10].

Median. A measure of the center of ordered observations, separating the lower and upper halves of the distribution.

Mixed-effects model. A statistical model combining fixed effects for systematic differences with random effects for variation between groups, such as participants [11].

Model Context Protocol (MCP). A protocol through which servers expose capabilities to clients. Its tool interface supports discovering and invoking named operations with structured arguments [24].

Monitoring. Observing an agent’s actions and their effects without being required to respond.

Normal mode / study mode. Caddie’s normal mode uses model-selected actions. Study mode follows predefined task specifications to compare oversight conditions under controlled action selection.

Odds ratio (OR). The ratio of outcome odds between two groups. Odds are $\pi/(1-\pi)$, where π is the outcome probability.

Oversight. The mechanisms through which a user can follow and govern an agent’s execution, including monitoring, confirmation, and intervention.

***p*-value.** The probability of a test statistic at least as extreme as observed, assuming the null hypothesis and the other statistical model assumptions [36].

Posterior distribution. The probability distribution of model parameters after incorporating the observed data and prior assumptions.

Prior distribution. A probability distribution expressing assumptions about model parameters before incorporating the data being analyzed.

R-hat ($\hat{R}$). A sampling diagnostic comparing variation within and between chains. Values near one indicate agreement between chains but do not establish that the model is appropriate [34].

Random intercept. A group-specific shift in a model’s baseline, here allowing participants to differ across their repeated observations.

Raw NASA-TLX. The unweighted mean of six NASA Task Load Index ratings, used here to summarize perceived workload [25].

Replay guidance. In Caddie’s normal mode, guidance derived from an earlier interaction trace. It informs model-selected actions without automatically executing the recorded sequence.

Retrieval-augmented generation (RAG). Supplying retrieved material as additional context when a language model generates its response [22].

Sensitivity analysis. Repeating an analysis under alternative assumptions or specifications to assess how its conclusions depend on those choices.

Shared autonomy. The thesis’s term for dividing task authority between an agent that can act and a user who retains defined control opportunities.

Standard deviation (SD). A measure of how widely observations vary around their mean, expressed in the same units as the observations.

Study fixture. A purpose-built study application with controlled content and resettable state, used to reproduce task conditions.

Task criticality. The potential consequences of an incorrect task outcome, represented in this study by predefined lower- and higher-criticality task sets.

Token. A unit into which text is divided for model processing. A token need not correspond to a complete word [10].

Tool call. A structured request naming an available operation and supplying its arguments for execution by software outside the language model.

UI Automator. Android testing tools for inspecting interfaces and simulating user interaction [6].

Within-participant design. A study design in which the same participants experience all compared conditions, allowing comparisons within each person.

1 Introduction

This chapter establishes the problem and scope of the thesis. It first motivates the need for user oversight in large language model (LLM)-based smartphone agents and identifies the research gap. It then states the research questions, summarizes the approach and contributions, and outlines the remainder of the thesis.

1.1 Motivation and Problem

LLM-based smartphone agents combine a language model that interprets a user’s request and proposes actions with software that executes those actions across applications. AndroidWorld comprises 116 tasks across 20 Android applications, but its strongest evaluated baseline completed only 30.6% of the tasks [29]. Reliable mobile task completion therefore cannot yet be assumed.

Such tasks are not limited to retrieving information: they can create or modify persistent application and device state, for example by sending a message, creating a calendar entry, or changing a system setting [29]. Errors in these actions can therefore leave effects that persist beyond the individual interaction step.

Delegating a task does not remove the need to oversee its execution. Stepwise confirmation can limit error propagation but adds repeated interaction. End-only confirmation reduces interruptions but can make recovery more costly after later actions depend on an error [42]. User involvement can help address imperfect plans and execution errors but can also increase cognitive load [17]. The resulting design problem is not simply whether a person should remain involved, but how that involvement can preserve meaningful control without turning automation back into manual work.

1.2 Research Gap and Objective

Prior work assigns user involvement to different stages and interfaces. One simulated daily-assistant experiment varied involvement during planning and execution [17]. A separate comparison examined intermediate confirmation and confirmation only at the end [42]. In mobile settings, DroidRetriever exposes progress and permits interruption or takeover during cross-application information seeking. AgentLens instead varies the visual modality shown when user interaction is needed [8, 19].

The systems and empirical studies reviewed in Chapter 2 do not compare three levels of required oversight for controlled smartphone tasks that change application or device

state. They also do not keep the running action narration available in every condition. This leaves open how different confirmation requirements affect objective interaction effort, subjective workload, perceived control, the perceived ability to intervene in time, and participant-reported detection of introduced errors.

This thesis addresses that question through Caddie, an LLM-based smartphone agent that executes state-changing tasks across applications while making its actions visible and allowing intervention. The study uses controlled local applications and preconfigured device states rather than participants' personal accounts.

The three conditions vary when participants must confirm task-progressing actions. C1 requires confirmation before each configured task-progressing step, C2 once before the requested change takes effect, and C3 no mandatory confirmation while intervention remains possible. In the within-participant study, every participant experiences all three conditions and completes tasks of lower and higher criticality, meaning the possible consequences of an incorrect outcome.

A controlled error is a predefined incorrect task parameter inserted into an assigned trial. Error detection is coded from post-task anomaly reports that identify the assigned error, and the reported stage is retrospective rather than directly observed. Objective interaction effort comprises required participant decisions and their cumulative response time. Study mode follows predefined nominal action plans, so the comparison evaluates the oversight design rather than the reliability of an unconstrained model.

1.3 Research Questions

The thesis addresses the following two research questions (RQ1 and RQ2):

- RQ1** How do different levels of required oversight affect objective interaction effort, subjective workload, perceived control, and the perceived ability to intervene in time when people use an LLM-based smartphone agent?
- RQ2** How does participant-reported detection of deliberately introduced errors vary between low- and high-criticality smartphone tasks and among the three levels of required oversight, in terms of detection probability and reported stage?

1.4 Approach and Contributions

The thesis combines technical evaluation, system development, and a controlled human-computer interaction (HCI) study. Technical experiments examine how language models use Caddie's structured operations to execute smartphone tasks and identify recurring failure patterns within the recorded configurations. These findings motivate an oversight layer that exposes the current action and supports confirmation, temporary pausing, action-boundary correction, and stopping subsequent action dispatch. In the

user study, the running action narration remains available, and predefined nominal action plans guide execution while the level of required oversight varies.

The work makes three contributions:

1. technical evidence about capability limits and recurring failure modes in documented Caddie configurations.
2. the design and implementation of Caddie’s oversight mechanisms for consequential cross-application tasks.
3. a controlled comparison of three oversight levels across tasks of lower and higher criticality. It quantifies the interaction effort required by confirmation and tests planned contrasts for workload, perceived control, and timely intervention. The results did not establish a workload difference or the full predicted ordering for perceived control and timely intervention. The analysis did not establish higher reported error detection with more frequent confirmation. Different preferences across task consequences motivate review opportunities that vary with those consequences.

1.5 Thesis Structure

The remainder of the thesis is organized as follows. Chapter 2 reviews LLM-based mobile agents and research on human oversight, transparency, and steerability. Chapter 3 describes Caddie’s architecture, execution modes, oversight mechanisms, and technical validation. Chapter 4 presents the controlled study design, and Chapter 5 reports the results for RQ1 and RQ2 alongside exploratory findings. Chapter 6 interprets the findings, derives design implications, and discusses limitations. Chapter 7 concludes the thesis and outlines future work.

2 Background and Related Work

Understanding Caddie’s oversight design requires two perspectives: how a large language model (LLM)-based agent acts on a smartphone and how a person can remain involved in that execution. The technical foundations of language-model agents and mobile graphical user interface (GUI) operation provide the basis for discussing human control and oversight. These perspectives are then used to compare the closest systems and empirical studies and to delimit the research gap addressed by this thesis.

2.1 From Large Language Models to Interactive Agents

Many large language models (LLMs) are neural networks trained on large text corpora to generate text one token at a time. A token is a text unit used for processing. During inference, when the trained model produces an output, it assigns probabilities to possible next tokens from the preceding context [10, 35]. PaLM exemplifies a decoder-only Transformer that generates each token from prior context without a separate encoder [10]. The relevant distinction here is between proposing an action and carrying it out. An LLM can generate a structured action, but other software must interpret and execute it [31, 39].

An interactive agent supplies this execution layer and returns observations from the environment to the model. This produces a repeated observation and action loop in which an action is selected from the current context, executed externally, and followed by an observation that can inform the next decision. ReAct (Reasoning and Acting) is a prompting method that interleaves model-generated reasoning traces with external actions and subsequent observations [39]. It illustrates one form of the observation and action loop. Not every agent uses ReAct or exposes its reasoning traces.

Within this loop, a tool is a callable operation exposed to the model by the surrounding system. Toolformer’s language model learns to generate API calls within its text. External tools execute these calls, and their responses become part of the context for subsequent text generation [31]. The evaluated APIs cover calculation, question answering, search, translation, and calendars rather than smartphone control. Nevertheless, the same division of responsibility applies to a mobile agent: the model produces a tool call, an external component executes the requested operation, and the resulting observation can inform a later decision. Caddie’s smartphone-specific tools and execution architecture are described in Chapter 3.

2.2 LLM-Based Agents for Mobile Graphical User Interfaces

Acting through a mobile graphical user interface requires an agent to solve two coupled problems: it must select an operation that advances the user’s goal and identify where that operation applies in the current interface. A command such as a tap is not actionable until it is associated with an interface element or screen position. Mobile-agent design therefore depends on both the observation used to represent the interface and the action interface used to affect it.

On Android, one possible source of structured interface information is the accessibility framework. Android’s documentation characterizes an accessibility service as a specialized assistive tool. When configured with the required capabilities, such a service can inspect a hierarchical representation of screen content and perform actions such as clicks, scrolling, and global navigation [3]. AndroidWorld illustrates how a research environment can use this representation. It includes M3A (Multimodal Autonomous Agent for Android), a baseline agent evaluated on the benchmark. AndroidWorld exposes a full-resolution screenshot and an accessibility-based UI tree. Its action interface supports tapping, swiping, text entry, navigation, and a limited set of function calls. Depending on its configuration, M3A uses the tree alone or combines it with an annotated screenshot. After each action, M3A observes the interface state again, and a summary of the result becomes part of the history used for later action selection [29]. This design is specific to AndroidWorld and M3A, not a required architecture for all mobile agents.

Whereas AndroidWorld is primarily a benchmark environment, AppAgent is a specific agent framework. At each step, AppAgent receives a screenshot and an Extensible Markup Language (XML) description of interactive elements. Numbered element identifiers connect the structured description to the screenshot. The agent then selects from actions including taps, long presses, swipes, text input, and back navigation. This design operates through the graphical interface without requiring application-specific backend functions [40]. The paper presents this as a route to broader app coverage, but the mechanism and the authors’ evaluation do not by themselves establish universal compatibility, security, or privacy.

MobileWorld broadens mobile-agent evaluation beyond graphical interaction alone by adding MCP-augmented tasks. In the Model Context Protocol (MCP), servers expose named tools with input schemas, and clients can list and invoke these tools using structured arguments [24]. Of MobileWorld’s 201 tasks, 40 are MCP-augmented. These tasks connect tool results to later graphical actions: in one example, the agent retrieves a repository’s commit history through a GitHub MCP tool, composes a summary, and sends it through the native email application. The integrated servers also include mapping tools for route planning and location search [20]. MobileWorld therefore demonstrates one benchmark design that combines external service calls with Android actions. Although it also includes clarification tasks with an LLM-simulated user, it evaluates agent performance rather than participant oversight.

2.3 Human Control, Automation, and Shared Autonomy

Automation is not a single system-wide setting. An established framework separates it into four functions: acquiring information, analyzing it, making a decision or selecting an action, and implementing that action. Each function can be automated to a different degree, from manual to fully automatic [27]. For a smartphone agent, these functions correspond to observing the interface, interpreting its state, selecting the next operation, and performing the selected tap, swipe, or other input. This thesis uses the framework to analyze the smartphone agent, although it was not originally developed for mobile LLM agents. The framework helps identify where the agent acts independently without labeling the entire system as simply manual or autonomous.

Human control refers to the user’s ability to direct, approve, or stop the system. Such control and computer automation need not occupy opposite ends of a single scale. A two-dimensional human-centered artificial intelligence framework treats them as separate dimensions and allows a design to combine high levels of both [33]. For a smartphone agent, reducing the number of manual actions therefore does not require removing every user decision or means of redirecting execution. The framework offers a design perspective, not evidence that a particular interface gives users high control or that they will experience it as such.

The related term *shared control* has a narrower meaning in parts of the human-robot interaction literature. In that narrower sense, it refers to a human and robot working together during the same ongoing cycle of observing and acting to produce a plan, decision, or control action. This definition excludes traded control, where the right to make the next control decision passes between the human and the automated system at different times [1]. It therefore does not cover every arrangement in which the system acts independently under human supervision or the person intervenes only at discrete points.

This thesis uses *shared autonomy* more broadly. The agent can perform actions while the user retains defined opportunities to authorize continuation, interrupt execution, or redirect it. Unlike the narrower robotics definition of shared control, this usage does not require the user and agent to contribute at the same time to a plan, decision, or action. In the terminology used throughout this thesis, *automation* refers to the extent to which a function proceeds without immediate user input, *oversight* to the ways in which the user can follow and govern execution, and *shared autonomy* to how authority over the task is divided between the agent and the user.

2.4 Oversight, Intervention, and Error Detection

In this thesis, oversight is the umbrella term for mechanisms through which a user can follow and govern an agent’s execution. Monitoring means observing the agent’s actions and their effects without being required to respond. A confirmation point is a system-defined pause at which the user must approve or reject continuation. An intervention

occurs when the user actively changes the course of execution, for example by rejecting an action, stopping the agent, or providing a correction. These mechanisms are related but not interchangeable: visibility supports monitoring but is not itself an exercise of control, confirmation creates a required decision, and voluntary intervention depends on the user noticing a reason to act.

Figure 2.1 arranges these distinctions around a generic observation and action loop. Monitoring connects the user to observations and action effects, confirmation inserts an explicit decision boundary between action selection and execution, and intervention allows the user to halt or redirect execution. The locations are schematic because systems can expose these mechanisms at different moments and in different combinations. The figure is therefore a conceptual model, not a description of Caddie’s implementation or experimental conditions.

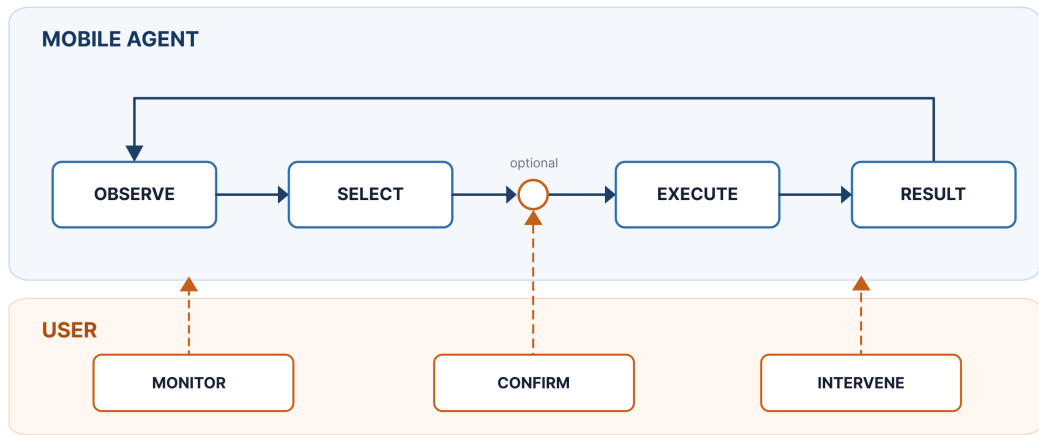


Figure 2.1: Generic points of user involvement in a mobile GUI agent’s observation and action loop. The framed regions separate the agent process from user involvement. The empty circle marks a possible confirmation point between action selection and execution.

The number and timing of confirmation points affect how often the user must respond and whether an error can be stopped before later actions build on it. In a study of when users should check an agent’s progress during multi-step tasks, Zhou et al. [42] contrast two extremes. Confirmation after every step can limit error propagation but adds substantial interaction overhead. Confirmation only at the end reduces immediate user involvement but can increase the cost of errors and re-execution. Their formative study also identified a recurring pattern of confirmation, diagnosis, correction, and redo: after a final result did not match their expectation, participants typically reviewed the execution history, located the erroneous step, supplied a correction, and waited for the agent to redo the remaining execution [42]. Confirmation therefore involves a trade-off: more checkpoints can limit how far an error spreads, but they also interrupt the user more often. Intermediate checkpoints can reduce how much work must be reviewed and repeated after an error. This work concerns agent-led desktop tasks and optimizes

interaction time under explicit assumptions. It does not establish the best confirmation placement for Caddie or for state-changing smartphone tasks.

Having an intervention control available does not guarantee that a user will recognize when it is needed or be ready to resume the task. This difficulty is known as the *out-of-the-loop performance problem*. A 1995 human-automation experiment by Endsley and Kiris examined this problem in a simulated navigation task. Participants who had previously worked with automation took longer to perform manually after the expert system failed than participants who had worked manually throughout. Level 2 situation awareness captures an understanding of what the system state means. Participants in the fully automated condition scored lower on this measure than those in the manual condition. The intermediate automation conditions produced scores between these two conditions [12]. The experiment did not involve an LLM or a smartphone interface, so it does not predict the outcome of Caddie’s oversight conditions. It instead shows why the availability of a control must be distinguished from a user’s readiness to recognize a problem and use that control.

Intervention behavior alone cannot establish whether a participant noticed an error. The study therefore considers two outcomes separately: whether participants intervened and whether they reported noticing an intentionally introduced error. Detection and its stage are assessed retrospectively after each task. These self-reports indicate whether participants said that they noticed an assigned error and which broad stage they selected. They do not establish the exact moment of recognition. Section 4.6.2 defines the coding rule and the role of runtime evidence.

2.5 Closest Systems and Empirical Studies

The comparison is organized around three questions. First, does a system operate mobile graphical interfaces or a different environment? Second, does its task end in information presentation or change the state of an application or device? Third, how can the user monitor, confirm, or intervene during execution? A system or study can be close to this thesis on one question without addressing the others. Accordingly, the discussion distinguishes implemented systems, empirical studies, preprints, and vendor documentation instead of treating every item as equivalent evidence. Commercial mobile assistants are included to describe the surrounding deployment context, but they do not substitute for peer-reviewed evidence about the study conditions.

DroidRetriever is a close comparison on mobile execution and user involvement. It accepts a spoken or typed information request and splits it into subtasks across applications, search terms, and pages. A UI copilot then navigates to relevant screens and captures screenshots. A separate report-synthesis module uses these screenshots to produce a report whose claims link back to them. During navigation, a dashboard shows subtask progress and live exploration traces. Users can interrupt or take over. The system pauses for intervention at privacy-sensitive screens or high-risk actions [8].

These mechanisms make DroidRetriever a design precedent for combining visibility with intervention during cross-application mobile work. Its evaluated goal is information retrieval followed by report synthesis, not task completion that changes the state of an application or device. The evaluation also does not manipulate confirmation frequency. This design demonstrates how mobile-agent execution can be exposed and steered without comparing the effects of stepwise confirmation, a final checkpoint, and voluntary intervention.

AgentLens addresses a different part of the same design space. Foreground execution exposes the agent’s application interactions but occupies the display. Background execution permits other phone use but provides little visual awareness. AgentLens runs an unmodified target application on a separate virtual Android display that is not shown to the user. When the agent needs to notify or question the user, a companion application shows selected information in an overlay above the application the user is currently viewing instead of returning permanently to full-screen execution. The prototype still needs a connected computer and Android Debug Bridge (ADB) to launch third-party applications on this virtual display. It does not demonstrate standalone deployment on an ordinary phone [19].

AgentLens can reuse the application’s interface or generate a new one for the current exchange. The two options that reuse the interface differ in how much of it they show: *Full UI* retains the complete screen and its spatial context, whereas *Partial UI* limits the view to the relevant region. *GenUI* instead reorganizes the interaction into a generated interface, which the authors position for lower-risk situations. These three visual modalities are selected at individual points of user involvement, so the presentation can change during a task [19].

Figure 2.2 shows how these three presentation forms appear in the AgentLens prototype.

This adaptive choice is motivated by the authors’ third formative study. Ten participants ranked the three modalities within each of six scenarios, producing 60 first-place ratings. Full UI and GenUI each received 19 first-place ratings, while Partial UI received 22; preferences also varied with the scenario. No single presentation form was preferred across all tested scenarios [19]. These results do not answer how often the agent should request involvement. The main AgentLens comparison asks how to display predefined moments when the user must interact, not how often those moments should occur. DroidRetriever instead focuses on showing progress and allowing takeover during mobile information seeking. Both systems provide design precedents for mobile user involvement, but neither compares stepwise confirmation, a final checkpoint, and voluntary intervention. AgentLens also remains an April 2026 preprint, so its findings are treated as provisional evidence.

Commercial mobile assistants illustrate product-specific forms of user control. Google’s February 2026 early preview describes visible task progress and intervention or stopping for selected background tasks in a virtual window [14]. Samsung describes defined cross-application actions that can retain a final user-controlled action, such as sending a prepared message [30]. Apple’s June 2026 announcement describes screen-aware,

2 Background and Related Work

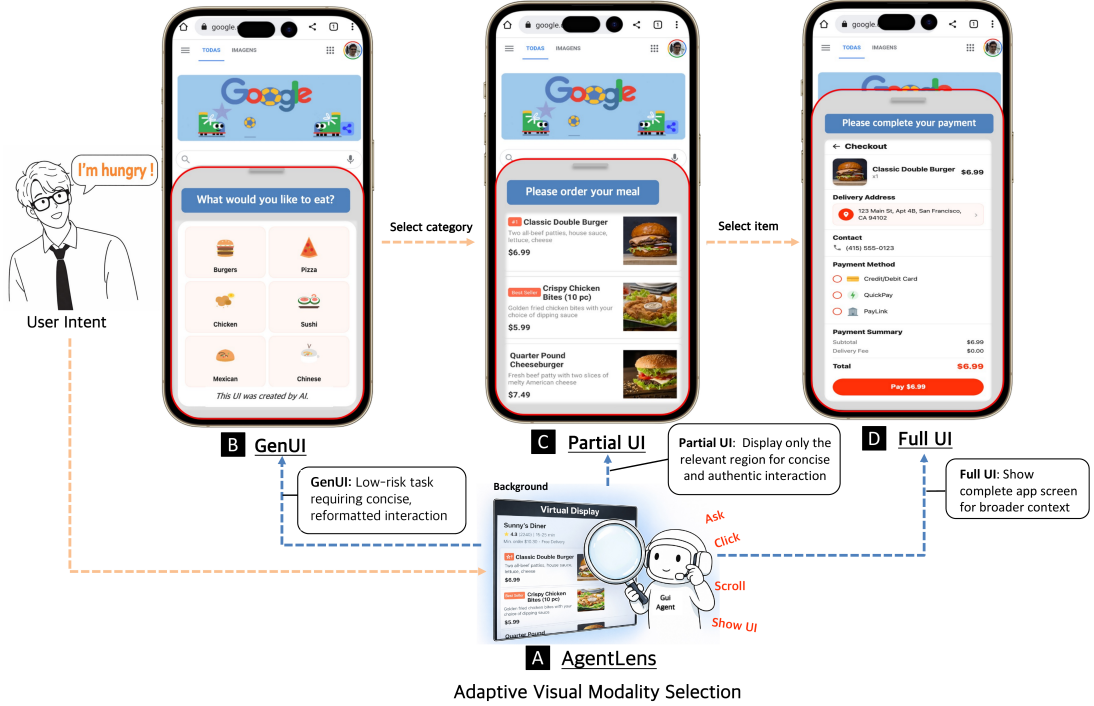


Figure 2.2: AgentLens presents a generated interface, a task-relevant region from the underlying application, or the full application screen at selected interaction points. Reproduced from Figure 1 by Kim et al. [19] without visual modification other than scaling. The source is licensed under CC BY-NC-SA 4.0.

system-defined actions in developer testing [7]. These vendor materials describe product features or plans rather than independent evidence of general availability, arbitrary GUI navigation, or the effectiveness of the controls.

Empirical work outside mobile GUI execution has examined both the stage and frequency of user involvement. A plan-then-execute experiment by He, Demartini, and Gadiraju involved 248 participants across six everyday scenarios with different levels of risk. The 2×2 design used four conditions. Planning was either automatic or involved the user, and execution was also either automatic or involved the user. During planning, involved users could edit, add, delete, or split steps in an LLM-generated plan. During execution, they could review each of the agent’s predicted actions and provide feedback or specify an alternative action [17].

User involvement was not uniformly beneficial. Participants could repair imperfect plans and execution actions, and involvement during execution produced more consistent benefits than involvement during planning. User edits sometimes reduced the quality of an initially sound plan. Involvement in both stages also produced higher cognitive load. No significant difference in subjective trust was found across the four conditions. These results come from tasks executed in a simulation through predefined backend APIs. The conditions combined involvement in planning and execution rather than isolating confirmation frequency during live smartphone interaction [17].

2 Background and Related Work

A separate investigation focused directly on confirmation frequency. Based on a formative study with eight participants, Zhou et al. developed a model that places confirmation points to minimize total task time. The model considers the time needed to confirm actions, diagnose errors, and repeat work, together with the estimated accuracy of the agent. A subsequent within-participant evaluation with 48 participants compared model-selected intermediate checkpoints with confirmation only at the end across simulated shopping, image-editing, and game tasks. Intermediate confirmation reduced mean task-completion time by 13.54% relative to end confirmation, with 81% of participants preferring the intermediate strategy. The time benefit depended on error position: it was largest for early errors, nearly absent for errors in the middle, and became a small cost for late errors [42]. This result demonstrates that checkpoint timing can affect total task-completion time and user preference in this simulated setting, but it was obtained with fixed execution times and accuracies. The comparison included only intermediate and end confirmation. It included neither an every-step condition nor a voluntary-intervention-only condition, and it did not execute actions through smartphone interfaces.

Two recent preprints examine oversight interfaces for computer-use agents more directly. The first, by Grunde-McLaughlin et al., reports three iterative studies with 12 participants each. Its final controlled evaluation compared a conventional action trace with a redesigned review interface. The redesigned interface organized the task’s requirements and the agent’s assumptions, linked them to relevant trace text and screenshots, and allowed access to the underlying websites. Verification was slightly faster on average overall, with a larger estimated time reduction when participants correctly rejected faulty outputs. The confidence intervals for both duration effects included zero. Judgment accuracy showed only a small estimated difference, while the largest estimated increase in confidence occurred when participants accepted an incorrect result [16]. The evaluation used completed question-answering runs and a Wizard-of-Oz study, in which researchers manually provided parts of the behavior presented to participants as automated. It did not vary confirmation frequency during live, state-changing execution.

The second preprint, by Chen et al., compares four broader oversight strategies with 48 participants in a within-participant design. *Risk-Gated* execution was autonomous by default and escalated steps classified as consequential. *Action Confirmation* required authorization before each action. *Supervisory Co-Execution* emphasized plan-level involvement while retaining a live trace and controls to pause, redirect, or take over. The *Structurally Enriched* condition combined plan-level and step-level engagement with risk labels and an option to open a more detailed review. The same Gemini-powered agent performed live tasks on real websites in all conditions. The six task designs embedded privacy leakage, prompt injection, or dark-pattern actions in higher- and lower-consequence scenarios [9].

The oversight strategy significantly affected whether a problematic action occurred. The analyses found no statistically significant differences among strategies in successful intervention after a problematic action became visible or in final attack success. Problematic actions occurred less often in the observed data for the two plan-oriented

strategies. The authors note that earlier plan revisions may have prevented these actions, rather than users successfully stopping them during execution. No strategy was uniformly best across workload, perceived control, trust, and usability. Trust showed the clearest context-dependent pattern [9]. The separation between exposure to a problem and correction after exposure is relevant to error detection in the present thesis. Because the four conditions alter several coordination and interface mechanisms together, the comparison does not isolate confirmation frequency and concerns live websites rather than smartphones. Both papers remain 2026 preprints, so their findings are treated as provisional evidence.

Table 2.1 condenses these comparisons and records the evidence type so that vendor descriptions are not read as independent performance evidence. Its columns distinguish the task setting, the form of user oversight, and the remaining difference from this thesis.

Table 2.1: Closest systems and empirical studies in relation to the oversight comparison in this thesis.

Setting and task	Oversight mechanism	Difference from this thesis
DroidRetriever <i>Peer-reviewed system paper</i> [8]		
Mobile information seeking and report synthesis.	Progress dashboard, interruption, takeover, and risk-triggered pauses.	Does not isolate confirmation frequency. Tasks gather information and synthesize reports.
AgentLens <i>Preprint</i> [19]		
Background mobile application operation.	Full, partial, or generated interfaces at selected interaction points.	Studies how interaction is presented, not when confirmation is required.
Gemini on Android and Galaxy devices <i>Vendor documentation</i> [14, 30]		
Early preview of selected tasks on supported devices and applications.	Progress view, takeover or stop, and selected explicit user actions.	Restricted availability and no independent evaluation of the described controls.
Siri AI <i>Vendor announcement</i> [7]		
Screen-aware, defined cross-application actions. Developer testing at the time of writing.	System-defined application actions. No evaluated oversight mechanism is reported.	Does not establish arbitrary GUI navigation or compare oversight designs.
Plan-Then-Execute <i>Peer-reviewed user study</i> [17]		
Six simulated everyday scenarios using predefined backend APIs.	User involvement during planning and/or execution.	Combines planning and execution involvement rather than isolating required confirmation frequency during live smartphone execution.
When Should Users Check? <i>Peer-reviewed user study</i> [42]		
Simulated agent-led desktop tasks.	Model-selected intermediate checkpoints or confirmation at the end.	Includes neither a voluntary-intervention-only condition nor live smartphone GUI execution.
Overseeing Agents Without Constant Oversight <i>Preprint</i> [16]		
Completed computer-use question-answering runs.	Linked requirements, assumptions, action traces, and screenshots support review.	Studies presentation of action history rather than required confirmation frequency.
Comparing Human Oversight Strategies <i>Preprint</i> [9]		
Live web tasks with embedded problematic actions.	Four packages combining planning, gating, review, and co-execution mechanisms.	Combines mechanisms rather than isolating confirmation frequency.

2.6 Synthesis and Research Positioning

As summarized in Table 2.1, prior work addresses several distinct oversight decisions. Mobile systems make execution visible, permit takeover, or adapt how selected interaction points are presented [8, 19]. Studies beyond mobile GUI execution examine whether users participate in planning or execution, where confirmation points are placed, how completed action records are presented, and how several oversight mechanisms work in combination [9, 16, 17, 42]. Changing what users can inspect is not the same as changing when they must make a decision. Treating both simply as human involvement would obscure the variable that each study manipulates.

Prior work therefore provides little controlled evidence about required oversight during smartphone-agent tasks that change application or device state. In particular, it remains unclear how the frequency and placement of confirmation affect interaction effort, workload, perceived control, the perceived ability to intervene in time, and participant-reported detection of deliberately introduced errors when action visibility remains available. Existing comparisons often change interface presentation, planning involvement, risk escalation, or trace structure together with when users must act. They consequently do not isolate required oversight from these other mechanisms.

This thesis studies this gap with three conditions: confirmation before each configured step that advances the task, one final confirmation before the requested change takes effect, and no mandatory confirmation while voluntary intervention remains possible. All three conditions show the running action narration. C2 additionally bundles pending actions and relevant values at its final checkpoint. The comparison therefore concerns these three implemented oversight designs and does not isolate confirmation frequency from the presentation of review information. The study combines each condition with low- and high-criticality tasks and predefined incorrect task parameters. This makes it possible to examine whether the effects of required oversight depend on task consequences.

Caddie provides the controlled research environment for this comparison rather than evidence of general autonomous reliability. RQ1 examines required participant decisions and cumulative response time, workload, perceived control, and the perceived ability to intervene in time. RQ2 examines the probability and participant-reported stage of detecting deliberately introduced errors across oversight conditions and task criticality. The resulting evidence is bounded to the studied tasks and interaction designs and cannot establish the safety or reliability of unconstrained smartphone agents.

3 Caddie: System Design and Implementation

This chapter presents the design and implementation of Caddie, the Android-based research system developed for this thesis. Normal mode handles open-ended requests through a language-model-driven agent loop. Study mode follows predefined trials so that different oversight requirements can be compared without model-selected action variation. The paths share the same Android application, participant-facing overlay, and Accessibility-based observation and action interface. The overlay remains visible above other applications, shows the current status and proposed action, and provides controls for confirmation, pausing, correction, and stopping.

A task begins when the user says “Hey Jarvis” from the Android home screen or another application. Caddie then enters a listening state, accepts the request, and makes its subsequent status and current action visible in the overlay. The later oversight section explains the confirmation, intervention, correction, and stopping controls that can interrupt this flow.

The chapter derives the system requirements from the research questions and then explains the phone-owned runtime boundary and the two execution modes. It next describes how normal mode uses task guidance, built-in and external tools, Android observations, action verification, and intervention controls. The chapter then presents the controlled study runtime. Finally, it separates development-stage evidence from current technical validation and documents the system’s reproducibility limits. Participant results are reported separately in the results chapter.

3.1 Design Goals and Requirements

RQ1 and RQ2 translate into six system requirements. First, participant-facing action narration must remain available in all three experimental conditions. Second, the frequency of required confirmation must differ across the conditions. Third, the conditions must use the same planned action sequence for each task so that model-selected plans do not confound their comparison. Fourth, each task must require a state-changing operation in an application or on the device rather than end with information retrieval alone. Fifth, participants must be able to intervene in execution in every condition. Sixth, the system must retain an inspectable record of trial assignments, participant decisions, execution events, and technical outcomes. Together, these requirements support RQ1’s measures of interaction effort, workload, perceived control, and perceived ability to intervene in time, as well as RQ2’s participant-reported detection outcomes.

Chapter 4 justifies the task classifications and defines the measures and coding procedure.

The study varies required confirmation while keeping action narration and voluntary intervention available. C1 requests stepwise confirmation, C2 uses one final checkpoint before the requested change takes effect, and C3 requires no confirmation. Predefined trial sequences keep nominal action selection independent of the model. Scheduled incorrect task parameters create controlled opportunities for error detection. Chapter 4 defines the conditions, task allocation, measures, and counterbalancing. Sections 3.4–3.6 explain the different execution and verification contracts of normal and study modes.

3.2 Architecture Boundary and Execution Modes

The first Caddie prototype ran agent orchestration in a Python process on a development computer. It communicated with the smartphone through the Android Debug Bridge (ADB), which lets a development computer send commands to an Android device [2]. After each observation, the language model returned one action encoded as JSON. The AndroidWorld baseline and the released AppAgent implementation also use host-to-device arrangements [29, 40]. In Caddie, this arrangement meant that each run depended on a separately started host process, an authorized debugging connection, and coordination of state between the computer and the Android application.

Development work also showed that providing more raw interface nodes or screenshots did not by itself give the model a reliable action strategy. UI Automator provides tools for inspecting Android interfaces and simulating user actions [6]. In a controlled comparison on a Pixel 7, Caddie compared its on-phone Accessibility observation with the hierarchy of interface elements exported by the UI Automator shell command `adb shell uiautomator dump`. In the tested states, the shell hierarchy omitted Caddie’s visibly blocking overlay and exposed a covered application control. The Accessibility observation retained the window order and marked that control as not visible. This single-device comparison does not establish that Accessibility is universally superior to UI Automator. It shows that the shell observation could disagree with the interface available to a Caddie user. Together with the host process’s operational dependencies, these observations motivated moving runtime control onto the phone and using Accessibility as the production source for interface observation and action. Section 3.7 returns to the comparison and its limitations.

The Android application now owns task input, routing, the agent loop, retrieved context, built-in tools, oversight, the overlay, Accessibility-based execution, persistent run and study records, and the controlled study runtime. The language model that generates Caddie’s responses runs on a separate project-controlled computer. Using the trained model to generate a response is called inference. The on-phone Model Gateway client communicates with that model service over HTTP. The model’s response is streamed back as it is generated. The gateway translates between Caddie’s internal model request and the service format, but does not execute smartphone actions. Caddie is phone-owned, but it is neither fully offline nor an on-device language-model

system. Section 3.7 examines two separate questions. First, can a candidate model complete Caddie’s tasks in the tested configuration? Second, could that model feasibly run directly on a smartphone? The candidate-model trials also used models running outside the phone. Their task failures therefore provide evidence about agent capability in those configurations, but not about the memory demand, latency, energy use, or thermal effects of executing the same model on the phone itself.

The Android runtime includes a client for the Model Context Protocol (MCP). MCP allows external servers to publish tools that a language model can discover and invoke [24]. Caddie’s client adds tools from available servers to the same registry as its built-in tools. Core smartphone observation and control use built-in Android tools. Server connections are therefore optional extensions for external services. They do not operate the phone. For study administration, the Android application runs an embedded HTTP server for the portal pages and its API. A web browser displays the task briefings and questionnaires and sends the entered information to the study phone. The browser does not execute participant tasks. Section 3.6 explains how the portal supports the study workflow and stores its records.

An on-phone router separates the two execution modes shown in Figure 3.1. Before the participant submits a request, the investigator activates the assigned trial for the current session. The trial’s trigger contract specifies task-specific required terms and accepted reference phrases. A matching request enters the study path, where the trial specification supplies actions, oversight points, controlled errors, and completion checks. The language model does not select those actions. During an active study session, a nonmatching request remains in the study path and prompts another attempt. It cannot enter the unrestricted normal agent. When no study session is active, a request enters normal mode. The agent loop then retrieves task-relevant guidance and asks the model to select a structured tool. The normal oversight policy permits ordinary actions and requires confirmation for irreversible, destructive, paid, or otherwise consequential actions. Section 3.5 describes this classification and its limitations. Normal mode is distinct from study condition C3, an experimental condition without mandatory confirmation. Both modes reuse the overlay and voluntary intervention controls, perform Android actions through Accessibility, check the resulting interface state, and persist their authoritative runtime records on Android.

Figure 3.2 shows Caddie’s application icon and its two setup states. While required access is missing, the setup screen reports the status of the Accessibility service, screen overlay, and voice input. Once all three are enabled, it changes to the ready screen. This ready state concerns Caddie’s access to Android. The same screen separately reports whether the configured model gateway is reachable. The screens are used to prepare and check Caddie, not to start a participant task. The task-start flow introduced above continues through the intervention controls in Section 3.5.

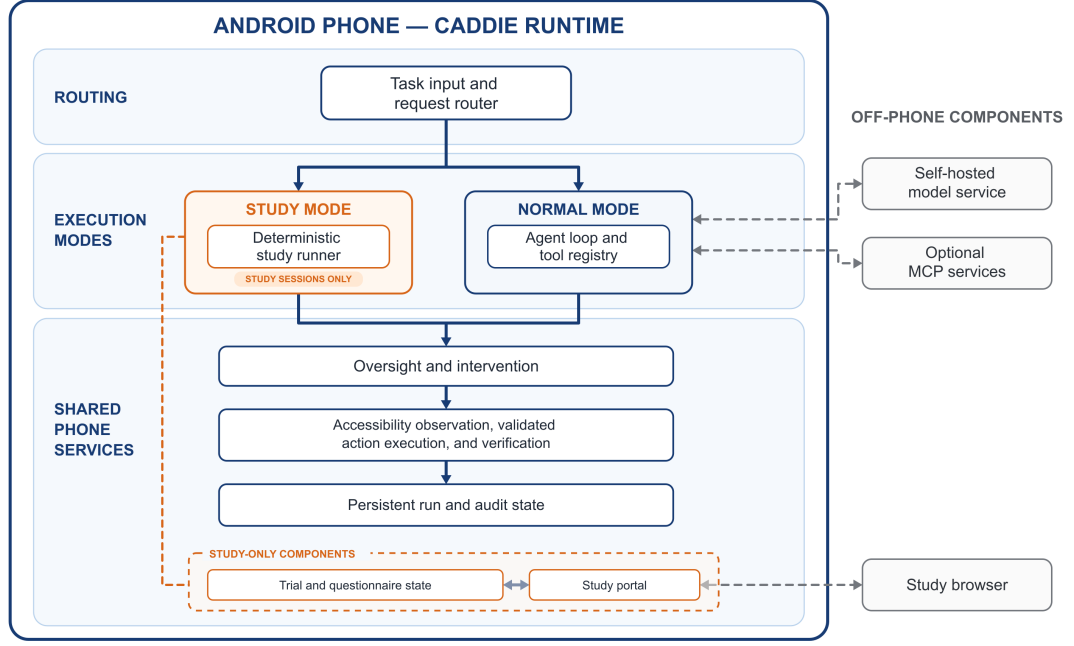


Figure 3.1: Phone-owned Caddie runtime. Solid blue arrows show control flow within the Android application, gray dashed arrows show communication across the phone boundary, and the orange dashed bracket links components used only during study sessions. Normal mode alone connects to the model and optional MCP services, while the client browser provides access to the phone-hosted study portal.

3.3 Normal-Mode Agent, Tools, and Context

Normal mode handles requests outside an activated study trial. The model selects the next operation, but it has no direct access to Android. Caddie separates model communication, available tools, and native execution so that the Android runtime retains control over validation, oversight, dispatch, and recording. This section explains the agent loop, its local and external tools, and the task guidance retrieved before model interaction.

3.3.1 Agent Loop and Model Gateway

For each normal-mode iteration, the agent loop reads the current run record. During an active run, this history remains in the Android application’s memory and contains the conversation, accepted tool calls, and their results. The loop combines it with the definitions of available tools to build the next model request. The Model Gateway converts the request into an OpenAI-compatible JSON format and streams returned text and structured tool calls back to the loop. The gateway is a network adapter between Caddie and the model service. It neither observes nor operates the smartphone. The evaluated normal-mode configuration uses the self-hosted `qwen3.6-35b-a3b` model,

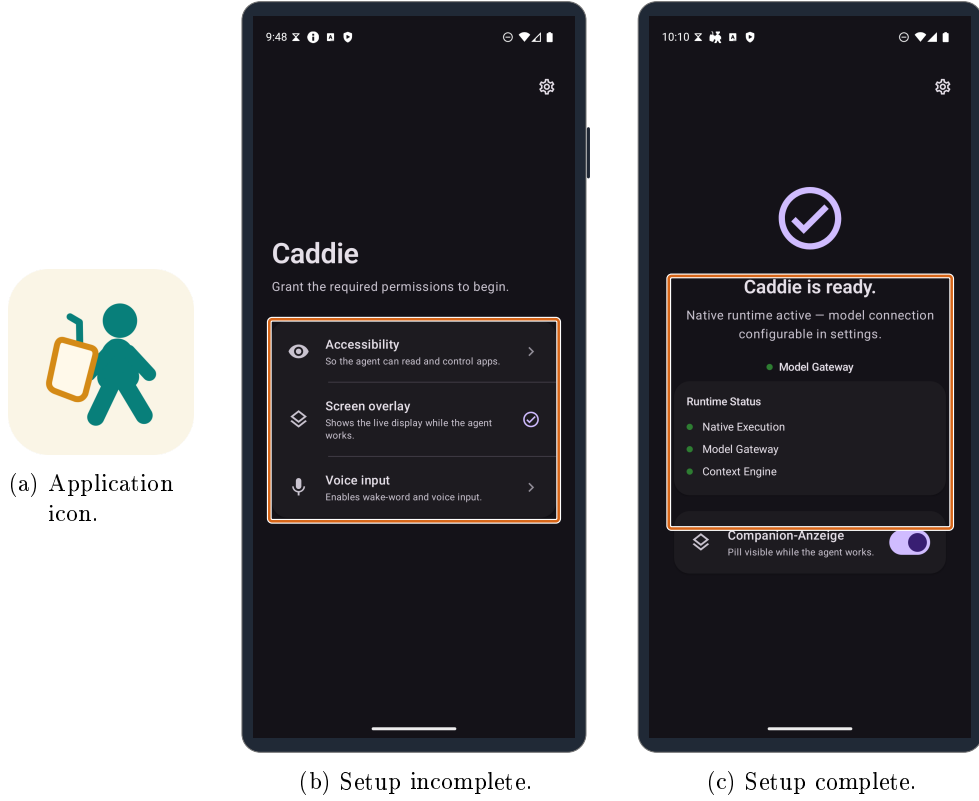


Figure 3.2: Caddie application icon and setup states on an Android emulator. The incomplete state shows Accessibility and voice input still pending. After Accessibility, overlay, and voice input are enabled, Caddie displays the ready state. This capture also shows a successful reachability check for the separately configured model gateway. The images contain no participant data. Schematic device frames retain the full display; orange outlines identify the relevant controls.

but the gateway can use another service that accepts the same request and streaming-response format. This names the selected deployment candidate, not an end-to-end success estimate for the final native normal-mode configuration. Section 3.7 explains the model selection and keeps task evaluations conducted during development separate from current integration checks.

The fixed system instruction defines a compact execution contract instead of task-specific procedures. It requires the model to treat interface text as data, never as instructions, observe after every Android action, briefly explain proposed actions, request missing essential information, and explicitly report success or failure. Task-specific skills and replay guidance are added separately. The instruction guides model behavior but is not a safety boundary. The Android loop independently enforces the single-tool-call rule, requires a registered tool route, applies oversight, and rejects completion without the required observation. Built-in tools also parse and validate their own typed arguments before they act.

Each iteration accepts exactly one structured tool call. A response with no tool call or multiple calls pauses the run instead of selecting an action implicitly. Before dispatch, Caddie preprocesses the call, looks up its route, applies the normal oversight policy and its risk-based confirmation decision, and records the accepted call. It then invokes the responsible tool, adds the result to the run record, and starts the next iteration. The model ends the run explicitly through either `caddie.complete` or `caddie.fail`. Caddie accepts completion only after observing the current interface following the preceding action. A model connection failure or an uncertain tool outcome pauses the run. The loop does not continue from an assumed state.

Caddie still uses the model service’s tool-calling format, but only to obtain one proposed operation at a time. Keeping the repeated loop in the Android application creates a control boundary between operations where Caddie can display the proposal, pause for intervention, incorporate a correction, apply confirmation, and verify the result before the next dispatch.

3.3.2 Built-in Tools and MCP Extensions

Caddie exposes typed tools with a name, description, and input schema specifying accepted arguments. The built-in registry maps each name to a native implementation for observing and operating the interface, asking the user a question, or ending the run. Smartphone actions execute locally so Caddie can oversee, verify, and record them.

The Model Context Protocol (MCP) lets clients discover and invoke tools exposed by external servers [24]. Caddie’s client converts available external tools into the same internal format as built-in tools. The registry rejects ambiguous names and routes each accepted call to its local component or remote server without changing the agent loop.

External MCP servers extend normal mode with service tools. Core smartphone observation and control remain local. During the controlled study, the deterministic study path executed predefined Android interface actions. None of its trial specifications required an external MCP service.

Local and remote calls pass through the same Android-owned route lookup, oversight, intervention check, and recording stages before dispatch. Built-in tools validate their typed fields locally. The current MCP client checks only that remote arguments form one JSON object. It does not independently enforce the server’s full published input schema. The shared loop can stop a call before dispatch, but it cannot cancel an operation that a tool has already started.

3.3.3 Skills, Retrieval-Augmented Generation, and Replay Guidance

AppAgent uses documents generated through autonomous exploration or human demonstrations to guide task execution [40]. AutoDroid retrieves app-specific navi-

3 Caddie: System Design and Implementation

gation knowledge from explored interface transitions [37]. These approaches motivate reusable application guidance, but their results do not establish its effect in Caddie.

Caddie uses selective app guidance to address two observed limitations of its earlier normal-mode agent. Keeping every task procedure in the fixed instructions sent with each model request, commonly called the system prompt, would include guidance that is usually irrelevant. Exact phrase matching also missed paraphrases and cross-language requests and could confuse similar triggers. Separate skills keep procedures inspectable and reusable, while retrieval selects guidance related to the current request. Replay guidance was intended to preserve useful navigation knowledge from a previous interaction without trusting old screen coordinates. Caddie’s evidence is mixed. Semantic retrieval improved skill selection in a small seven-skill pilot, but later generic explored hints did not improve end-to-end task completion. The mechanism selects and reuses context. It is not presented as a demonstrated improvement in agent reliability. Section 3.7 reports the corresponding experimental designs and results.

In Caddie, a skill is a bundled guidance document that describes when it is relevant, how a task can be approached, and what should be checked afterwards. Before sending the first model request for a normal task, Caddie searches for relevant skills in two ways. It compares words directly and uses semantic embeddings, which are numerical representations that help identify requests with similar meanings. The bundled `multilingual-e5-small` encoder creates these embeddings on the phone, allowing German requests and English skill descriptions to be compared without a separate translation step. It does not generate agent responses. If the encoder cannot be loaded, retrieval falls back to word matching. If context preparation fails entirely, Caddie sends the original model request without retrieved guidance.

Retrieval-augmented generation supplies retrieved text as additional context during generation [22]. Caddie uses this retrieval-before-generation pattern by adding a bounded selection of skill text to the request sent to the off-phone model. The text is reference material: it can change the model’s guidance but cannot authorize a tool call, bypass an oversight decision, or establish that an action succeeded.

A retrieved skill may link to replay guidance derived from an earlier interaction trace. During import, Caddie removes screen coordinates and other unstable element indices because they may no longer identify the same control in a later interface state. The remaining replay text tells the model to use a fresh interface observation and verify the resulting state. The production runtime treats this material only as guidance. It does not automatically dispatch the recorded action sequence. Activated study trials bypass this normal request-decoration process and follow their assigned deterministic specifications.

Figure 3.3 moves from the deployment view in Figure 3.1 to the sequence within one normal-mode iteration. A built-in Android action and an optional MCP call share the same route lookup, oversight, intervention check, and durable dispatch record. An MCP result returns to the agent loop without passing through Accessibility.

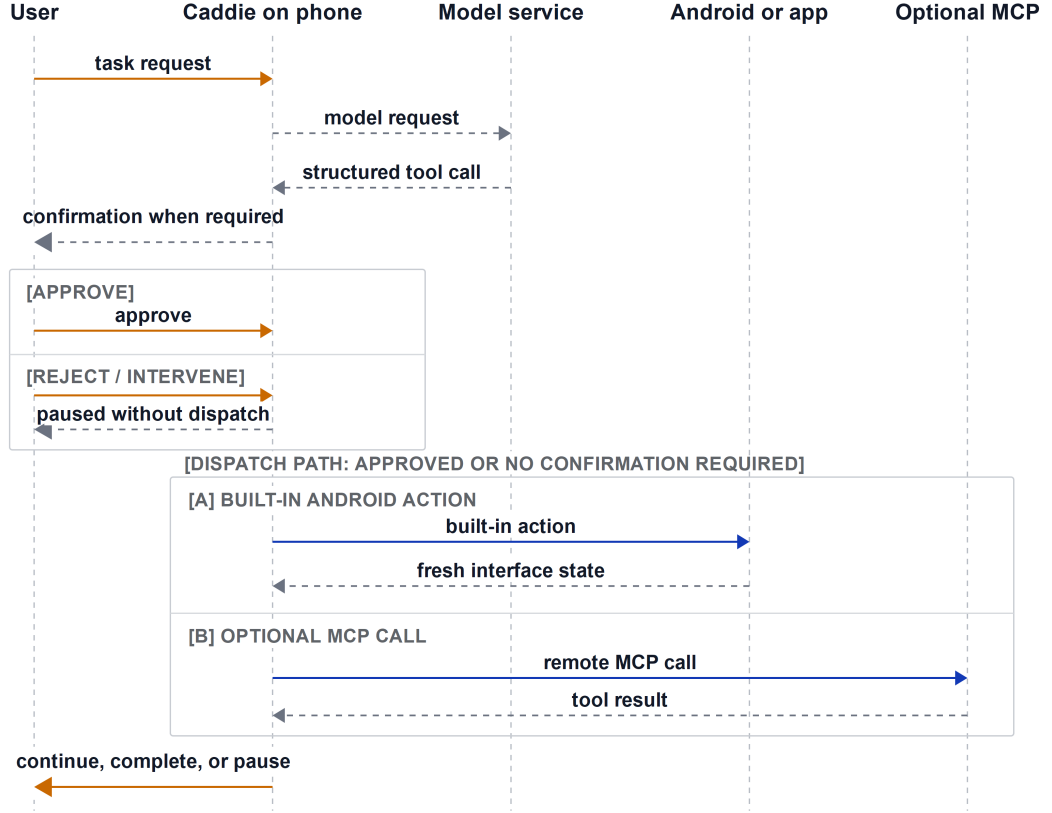


Figure 3.3: Sequence within one normal-mode iteration with two alternative dispatch routes. Caddie retains validation, oversight, dispatch, and recording on the phone. Built-in Android actions additionally use fresh interface observations for postcondition verification, while model inference and optional MCP tools remain network services. Rejection or intervention exits before either dispatch route.

3.4 Android Observation and Verified Action Execution

Normal and study modes act through native Android facilities but use different execution contracts. Android Accessibility is an operating-system service through which an authorized accessibility service can receive interface events, inspect structured elements exposed by applications, and perform supported interaction actions [3]. Caddie reads the current Android windows and traverses their interface elements to create an observation snapshot. For each element, the snapshot can contain its application package, resource identifier, visible text, accessibility description, element type, screen bounds, state, and supported actions. It also retains window order so that an element in a covered application is not treated as available to the user.

Figure 3.4 illustrates how a visible Android interface becomes a structured snapshot and how Caddie uses fresh snapshots to resolve and verify a normal-mode action.

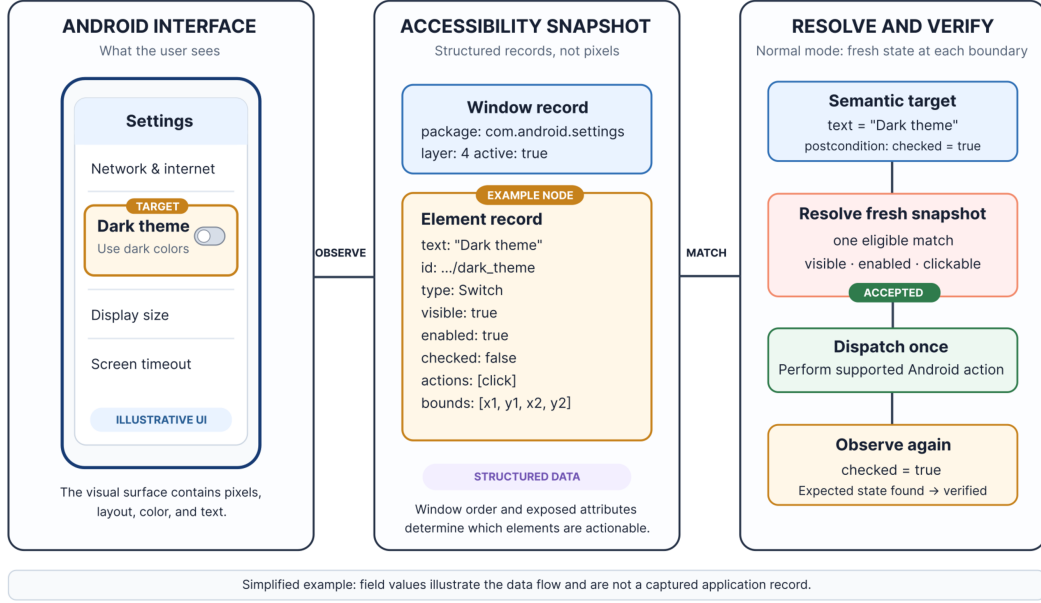


Figure 3.4: Illustrative accessibility snapshot and normal-mode target resolution. Caddie converts current Android windows into structured records rather than image pixels. Before dispatch, one eligible target must remain after checking visibility, enabled state, action support, and blocking windows. A later snapshot is checked for the expected postcondition. The displayed values are illustrative rather than a captured runtime record.

This structured observation is not a screenshot. It represents information that applications expose through Android’s accessibility framework, so visual details or labels can be missing. Caddie relies on this structured observation when executing an action because it identifies a current interface element and the operations that Android allows on it. Screenshots remain supplementary diagnostic evidence. Before dispatching a normal-mode interface action, Caddie obtains a fresh snapshot and resolves a semantic target from attributes such as its resource identifier, text, accessibility description, application, or element type. The runtime does not assume that this snapshot is complete. It rejects an action when the snapshot is incomplete, no semantic target matches, or eligibility checks leave zero or multiple targets. An eligible target must be enabled, visible, support the requested action, and not be blocked by a higher window such as Caddie’s confirmation overlay. This conservative policy prevents coordinate guesses when the current interface does not identify one safe target.

A resolved target is not proof that an action succeeded. For every normal-mode action that changes the interface, the model must also describe the interface state expected afterward. This expected state is called a postcondition. It is included in the same structured tool call rather than requested from the model later. A postcondition identifies an interface element and can specify whether it should exist, what text it should contain, or whether it should be checked or selected. Caddie rejects malformed tool arguments. Opening an application or web address uses separate verification. The ver-

ifier checks whether the expected destination application becomes the active or focused Android window.

Before physical dispatch, Caddie writes a unique run and attempt identifier to its persistent Android journal. It performs the action once and polls fresh Accessibility observations for the specified postcondition. The result is recorded as verified only when that condition is observed. If the platform rejects the action, the expected state does not appear, or the outcome remains uncertain, the run pauses instead of repeating the action automatically. This prioritizes duplicate-action safety. Repeating an uncertain message, payment, or deletion can be more harmful than requiring human review. Because the model supplies the normal-mode postcondition, successful verification establishes only that the specified interface state was observed. It does not independently prove that the postcondition fully represents the user’s goal. Study mode uses predefined task-specific rules for final trial verification, as described in Section 3.6. Section 3.7 evaluates the execution contract and its remaining limitations.

3.5 Oversight and Intervention

Both operating modes use the overlay to make the runtime’s current state visible above other applications. It distinguishes listening, thinking, acting, paused, completed, and failed states. While an action is in progress, it shows a human-readable description such as opening an application or selecting a labeled control. When Caddie requires the user to approve a pending action, the overlay shows the proposed action and provides a swipe-to-confirm control and a decline button. The overlay remains part of the Android window set observed by Caddie. The execution layer can therefore detect when the confirmation surface covers the target application and wait for the participant’s decision. While the surface is open, it receives touch input directly. A swipe on its confirmation control is not a voluntary intervention with the underlying application.

Figure 3.5 shows this task start and the main participant-facing oversight states. The first part shows the listening overlay, current-action narration, and the voluntary-pause display also available in C3. The continuation shows the C1 stepwise confirmation and the C2 final checkpoint, each with an enlarged view of the confirmation panel. The principal German interface labels translate as “Review action,” “Swipe to confirm,” “Reject action,” “Open latest whiteboard photo,” and “Save new note.” The images use synthetic test states rather than participant data.

Approval requires dragging the control through at least 75% of its available travel rather than tapping it once. The interface labels this interaction *swipe-to-confirm*. A touchscreen-input experiment found that unintended contacts differed from intentional input in timing, location, and movement [23]. Caddie uses the drag as a deliberate confirmation barrier intended to prevent a brief stationary contact from authorizing an action. The present study did not compare swipe and tap confirmation, so it does not establish that swiping is generally safer.

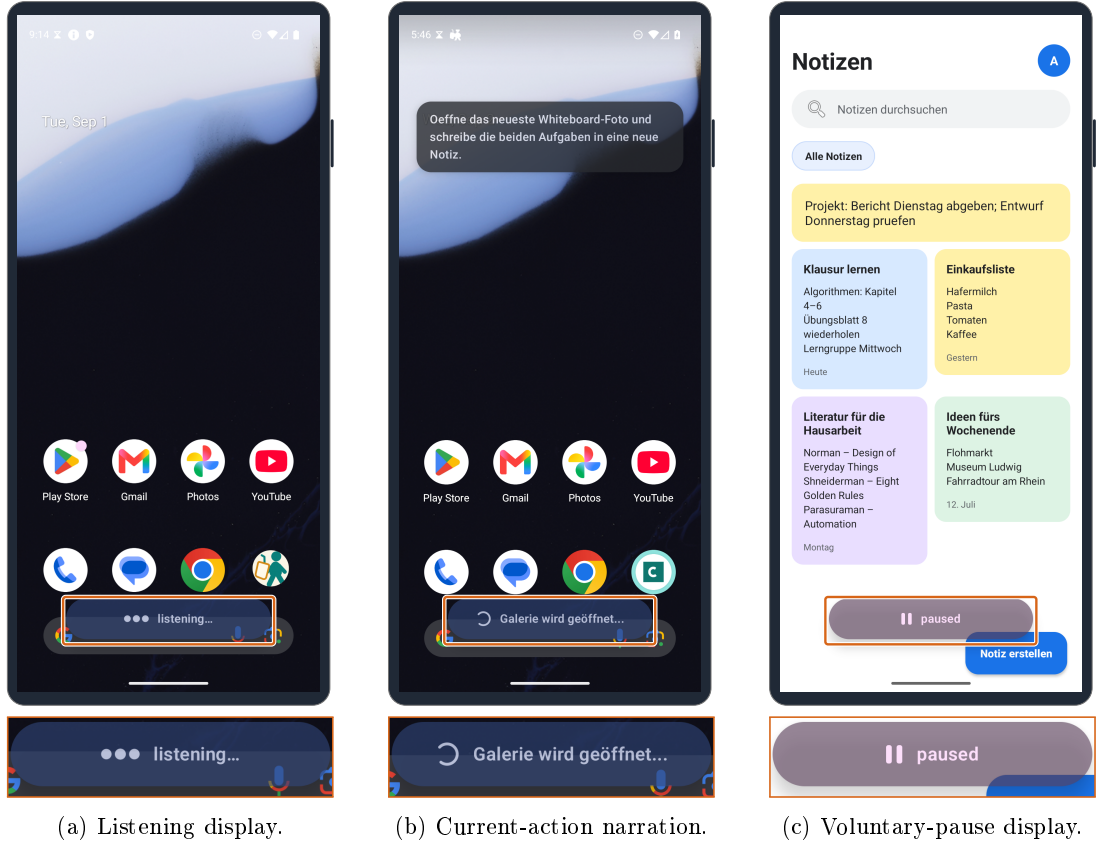


Figure 3.5: Task start and participant-facing oversight displays in illustrative test captures, without participant records. Orange outlines identify the overlay within the full display; enlarged details appear below. The listening and pause displays were activated directly for illustration. Device frames are schematic. Confirmation panels follow in the continuation.

In normal mode, confirmation depends on the proposed action, not a fixed experimental condition. A local rule-based classifier examines the tool name and its arguments. For actions directed at an interface element, it also uses the text and accessibility description of the matching element in the current observation. Ordinary actions can proceed automatically. Actions associated with sending, purchasing, deleting, changing access, or another consequential effect require confirmation. Caddie also requests confirmation when a click, long click, checked-state change, or text submission targets an element without an explicit human-readable label. The classifier does not rely on the language model’s own claim that an action is safe. Rules based on tool identities and interface labels cannot cover every meaning that an action may have in a particular real-world context. Section 3.8 considers this limitation again.

For MCP-discovered tools, the classifier uses tool-name heuristics to identify consequential operations and gates an otherwise unknown remote tool unless it matches an explicit read-only exemption. A server can misrepresent a mutating operation through

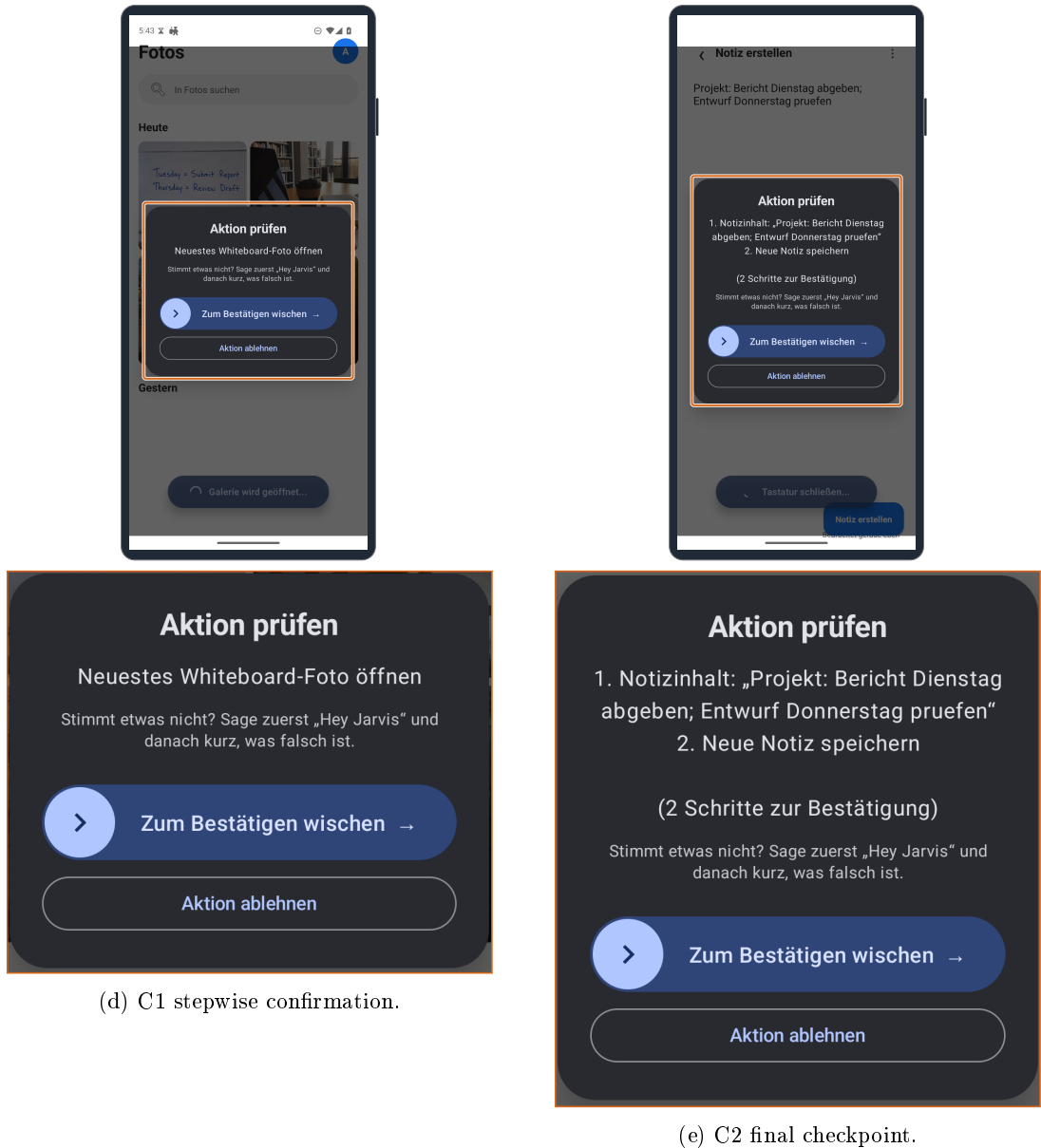


Figure 3.5: Task start and participant-facing oversight states (continued). Full displays and enlarged confirmation panels from the same captures. C1 requests approval of one configured task step; C2 summarizes the pending changes before the task’s commit step. Both panels were captured at running synthetic test-trial gates.

its name, so configured MCP servers and exemptions remain part of Caddie’s trusted configuration.

Normal mode is therefore not an implementation of C1, C2, or C3. Ordinary actions resemble C3 because they require no mandatory confirmation, but an action classified as consequential is blocked by an action-specific gate. This gate differs from C2’s single batched checkpoint before the task’s commit step. The model proposes the tool call

but does not decide whether confirmation can be bypassed. The Android-owned loop applies the local policy before dispatch and pauses the run unless the action is approved.

Study mode does not use this risk classifier to set mandatory gates. The activated trial specification sets the oversight condition. Under C1, Caddie requests confirmation before each configured step that advances the requested task. Interface-only preparation, such as focusing an input field or hiding the keyboard, can proceed without a gate. Under C2, Caddie presents one summary immediately before the commit step that makes the requested change take effect, for example by sending or submitting it. C3 adds no mandatory confirmation. Approval resumes execution. Declining a gate or allowing it to time out prevents the pending action. Chapter 4 gives the experimental rationale and full condition definitions.

Voluntary intervention remains available in every study condition and in normal mode. When a participant touches the display during an active run, Caddie prevents the next action from being dispatched while it determines whether the gesture is a tap or swipe. The pause is not shown until this classification is complete. A swipe is treated as an ordinary user gesture. Android delivers the original gesture to the visible application, Caddie releases the provisional hold, and no pause is reported. A tap produces a visible pause. If no further touch is detected, Caddie automatically resumes after a fixed quiet period of about two seconds.

Voice control uses a separate capture path. An on-phone wake-word detector starts capture of one utterance and holds the currently active run. Caddie delegates transcription to Android’s installed speech-recognition service. In normal mode, the language setting requests German or English recognition and model responses. Local commands support stopping, correcting, and clearing conversation context. Speech captured during an interrupted run is treated as a correction unless it matches another command. The transcript is bound to that run so that it cannot affect a later one. In normal mode, the correction is added to the next model request. Study mode remains fixed to German. A study correction is accepted only when it can be matched to the active assigned controlled error and then follows the predefined correction policy. A stop command prevents subsequent dispatch for the bound run. A recognition error releases the voice hold without creating an action. Because transcription is delegated to the installed recognition service, whether utterance audio is processed only on the phone depends on that service’s implementation and configuration. The normal agent can also receive text input. Figure 3.6 shows the language selector in the normal-mode runtime settings.

Caddie cannot reliably undo an action that Android has already accepted. Confirmation and pre-dispatch checks therefore remain necessary for consequential operations. The quiet period is used consistently across modes. Its duration was chosen during implementation and was not derived from a validated human response-time threshold.

3.6 Controlled Study Runtime and Instrumentation

The measured tasks use seven purpose-built local applications whose data can be reset to a known starting state: banking, calendar, gallery, mail, music, notes, and Telegram. The six tasks use combinations of these applications. The Maps/Messenger task additionally uses Google Maps together with the Telegram application. In Calendar/Do Not Disturb, the requested rule is represented inside the calendar application rather than by changing Android’s system-level Do Not Disturb setting. Seeded content and reset operations make starting states repeatable without using real financial or personal account data. For example, the banking task commits a simulated transfer inside Study Bank. These controls aid reproduction but limit how directly the observed behavior can be generalized to changing third-party applications and personal accounts.

Before a trial begins, the investigator loads one prepared assignment into the Android application. It specifies the participant identifier, task, oversight condition, and whether the trial contains a controlled error. Caddie then *arms* that exact trial and waits for the participant to state the task. A local study-routing component compares the utterance only with the predefined words and phrases that start the armed task. If it matches, the deterministic runner starts the assigned trial and executes the ordered steps from its specification. An unmatched utterance asks the participant to try again while study mode is active. It cannot fall back to the normal model path. Each specification is a versioned JSON document bundled with the application and validated when loaded. The runner does not ask the language model to select actions or add normal-mode skills or retrieved guidance. Fixing the nominal plan removes variation from model-generated plans across the three oversight conditions. Here, deterministic describes predefined nominal action selection, not an identical physical trace in every trial. The executed path can still branch when a state is already satisfied, a participant declines or corrects an action, a timeout occurs, Android rejects an action, or final verification fails.

Before the assigned trials, the portal can temporarily release the phone into normal mode for free practice. Participants can try the same voice input, narration, pause, correction, and stop controls without receiving a main-study task. Starting practice removes any armed trial and opens the Android home screen. Completing training returns the workflow to trial preparation. Practice uses the model-driven normal agent. The six measured trials use the deterministic study path described above.

Table 3.1 links one controlled value to its error variant, the irreversible commit, and the final technical checks. The complete specification also contains the participant briefing, trigger contract, narration, correction policy, timeouts, and remaining steps. The experimental assignment table supplies C1, C2, or C3 for each trial independently of the task specification.

The trial specification also defines action narration, confirmation points, an optional controlled error, the correction procedure, and final verification rules. A controlled error is a pre-specified wrong value inserted consistently into an assigned trial. Caddie applies the assigned variant to any applicable oversight presentation. When execution

Table 3.1: Selected fields of the bundled banking trial specification.

Specification field	Banking example	Runtime purpose
Task identifier	<code>task_banking_payment</code>	Selects the prepared banking workflow.
Controlled value	Correct amount 30 euros and error variant 80 euros	Keeps the planned value and assigned incorrect value together.
Correction policy	Return to the amount field before commit. Reject the correction after commit	Defines the permitted recovery boundary.
Commit step	<code>send_payment</code>	Marks the step that makes the simulated transfer take effect.
Final verification	Recipient, purpose, and amount must all match	Determines whether the requested task state was reached.

reaches the affected step, it records the error variant, affected step, correct value, and inserted value in the study database. Caddie records the audit event before the action is sent to the target application. The narration interval and a final opportunity to intervene occur in between. The event therefore shows only that the runner reached the controlled-error step with the incorrect value. It does not prove that the wrong action reached the application.

A spoken correction is processed only for the active trial and follows its predefined correction policy. Depending on that policy, the executor can return to an earlier step, perform an additional action that compensates for the incorrect action, or reject a correction after the commit has made the change irreversible. Once the action sequence ends, task-specific rules poll fresh Accessibility snapshots for expected or absent text, interface elements, and other defined postconditions. A verification rule can use up to 25 observations, with 150-millisecond pauses between observations. Each rule determines whether the observed state satisfies its required condition. Passing all checks produces a technical success. If verification fails while an assigned error remains uncorrected, the runner records an uncorrected-error outcome. Otherwise, failed verification produces a verification-failure outcome. This classification alone does not establish why a check failed. These technical outcomes do not determine whether or when the participant noticed the controlled error.

The audit stream assigns database timestamps to trial starts and completions, completed or already-satisfied steps, controlled errors, interventions, corrections, and verification results. A completed-trial event also contains the total trial duration. These records support reconstruction of event order and overall duration. For the final P02–P19 cohort, required confirmation responses were recorded during trial execution. Each response record links the presented C1 step gate or C2 final checkpoint to the participant’s decision and its response latency. These records provide the basis for the RQ1 measures of required-decision count and cumulative response time. Chapter 4 defines the resulting interaction-effort measures.

Post-task questionnaires record reported anomalies and, where applicable, what participants noticed, when they noticed it, and how they responded. The final P02–P19 dataset contains anomaly answers for all 108 scheduled tasks. Section 4.6.2 defines exposure and the coding of detection, non-detection, and indeterminate reports. Timestamped execution events can corroborate opportunity, reaction, and technical outcome, but cannot replace participant reports of noticing an error.

The study portal was developed to bring trial preparation, task briefings, and questionnaires into one workflow instead of using paper forms. It links each participant’s responses to the assigned trials and gives the investigator controls for preparing trials and releasing the corresponding participant pages. This connects study administration to the trial state recorded by the phone. Chapter 4 describes the instruments, task allocation, counterbalancing, and data collection procedure.

The portal’s web pages, styles, and scripts are bundled in Caddie’s Android application package. An embedded HTTP server serves them from the phone. It runs in an Android foreground service, which displays an ongoing notification while the portal is active. A browser on the investigator’s laptop displays the pages and sends entries and preparation requests to the phone’s API. The Android study runtime processes these requests and uses Room to store workflow state, questionnaire responses, trial markers, interview notes, and audit events in a local database. Research data and consent records have separate exports. The portal therefore needs no separate application server and is not part of the normal agent’s model, retrieval, or Android-action path.

3.7 Development Evidence and Technical Validation

Some evaluations were conducted before runtime control moved from the Python/ADB setup described in Section 3.2 to the Android application. This section separates those earlier engineering evaluations from validation of current implementation contracts. The evidence includes archived model trials, retrieval experiments, automated component and integration tests, and controlled runs on Android devices. The earlier evaluations explain design decisions, whereas current checks establish whether a stated contract was met in a tested configuration. Neither constitutes participant evidence or an end-to-end reliability estimate for the open-ended normal agent.

Table 3.2 summarizes each principal check and its interpretation boundary. The underlying reports and raw records are preserved with the submitted repository. They do not all describe the later native-runtime revision.

Table 3.2: Development evidence and technical validation overview.

Check	Tested setup and criterion	Result and limitation
Candidate models	Six models, six tasks, two runs per task on a Pixel 9a emulator; the report’s final-screenshot review, rather than completion calls, determined success.	Seven of 72 runs reached the target state in the manual screenshot classification. The labels were not independently or blindly produced. These runs used the earlier Python/ADB setup described in Section 3.2, so the count is not a current success rate.
Retrieved guidance	Seven-skill held-out matching pilot, followed by six device tasks with and without explored hints, three repetitions per configuration.	Top-ranked retrieval was 70% versus 20% in the pilot; the later evaluator labeled 12/18 runs successful in both configurations, using agent-verified rather than independently checked final states. Neither check evaluates the final corpus.
Execution strategy	Eight Android Settings tasks compared visible UI navigation with task-specific direct shortcuts. Replay was disabled; the early run used one repetition and the later run used two.	The early comparison recorded 7/8 versus 8/8 successful outcomes and 59 versus 19 seconds per task. The later comparison recorded 15/16 in both strategies and 22 versus 16 seconds. Both comparisons predate the move to the Android-owned runtime and use small task sets.
Android observation	Stable before/after captures on a Pixel 7 with Android 16, comparing the visible C1/C2 overlay boundary.	Accessibility retained the blocking window; the shell UI Automator dump omitted it. The comparison covers one device and selected states only.

3.7.1 Model and Retrieval Evaluations During Development

An early candidate-model matrix compared six self-hosted models, served from a separate computer, across six smartphone tasks with two runs per task. The tasks included changing display and connectivity settings, starting a timer, and performing a web search. The experiment used a Pixel 9a emulator and a host-controlled ADB execution setup. It is comparative evidence, not an estimate of the current Android-native runtime. The retained report records manual final-state classifications rather than treating the model’s completion call as evidence of success. Across the 72 runs, approximately 75% ended with a model-generated completion call, but only 7 were classified as reaching the requested state. The labels were neither independently produced nor blinded to the tested model. Valid screenshots remain for 67 of the 72 self-hosted runs. The remaining five runs have no usable screenshot, so the complete matrix cannot be rechecked from the images.

The selected `qwen3.6-35b-a3b` candidate had the highest success count among the self-hosted models, with 4 of 12 runs. The retained report includes wrong-application navigation, incorrect parameter handling, unchanged target controls, and declarations of success while the phone state remained unchanged. These failures motivated the

relative model choice and the requirement that Android verify an observable postcondition. Because the execution code, prompts, and device state subsequently changed, the counts are development evidence rather than a current success rate.

A separate reference run, recorded by its runner as using Claude Opus 4.7, used a different client against the same six task definitions and ADB-backed MCP interface. Manual screenshot inspection classified 11 of 12 runs as successful, compared with 4 of 12 for the selected self-hosted candidate. The differing clients and model configurations, small task set, and use of the earlier Python/ADB setup prevent a controlled cloud-versus-self-hosted comparison. The result records a large configuration-level performance difference without identifying its cause.

Retrieval was evaluated separately from task execution. In a seven-skill pilot, semantic retrieval returned the correct skill as its top-ranked result for 70% of held-out requests, compared with 20% for exact trigger matching. Its largest gains were for paraphrases and cross-language requests, while exact trigger matching remained better for its own literal phrases. The pilot used an earlier MiniLM embedding model and a manually assembled seven-skill corpus. Its results support combining semantic and lexical evidence, but they do not establish the accuracy of the final 76-skill corpus. A later device experiment compared six tasks with automatically collected navigation hints from earlier exploration either enabled or disabled, using three repetitions per task. Its evaluator labeled 12 of 18 runs successful in each configuration, with nearly identical turn counts and wall-clock times. These labels were agent-verified rather than independently checked against the final device state. The pilot supports improved matching. The later experiment found no completion benefit from the tested navigation hints. Neither experiment establishes the reliability of the current normal agent.

The final native normal-mode composition was checked separately from these earlier experiments. A focused Android run loaded the bundled skill corpus, the on-phone E5 embedding model, and linked replay guidance before streaming a real request to the self-hosted generative model. Component and integration tests also cover streaming model responses, typed local tools, MCP tool discovery and dispatch, retrieval fallbacks, and the separation of normal and study requests. Together, these checks show that the components are connected through the intended boundaries. They are not an end-to-end success benchmark for the normal agent. The controlled study does not depend on an MCP server.

3.7.2 Execution Strategies and Task Efficiency

Two pre-study engineering comparisons evaluated how direct shortcuts affected the execution of the same eight Android Settings tasks. The observable strategy navigated through the visible interface, whereas the fast strategy could use task-specific Android intents and deep links. Replay guidance was disabled in both strategies. These comparisons used the Python/ADB setup described in Section 3.2, not the C1, C2, and C3 oversight conditions used in the participant study.

3 Caddie: System Design and Implementation

In these historical reports, an agent turn is one iteration of the agent loop. In ordinary execution, each such iteration obtains a model response. For each strategy, the reported turns and times are unweighted means across the eight task-specific means; where two runs per task are listed, each task mean is first calculated across those two runs.

Table 3.3: Comparison of observable and fast execution strategies during development. “Agent turns” are historical agent-loop iterations; turn and time values are unweighted means across the eight Settings tasks.

Evaluation	Runs per task	Observable strategy	Fast strategy
Early phone run	1	7/8 recorded successes; 13.5 agent turns; 59 s	8/8 recorded successes; 3.9 agent turns; 19 s
Later in-process run	2	15/16 recorded successes; 5.8 agent turns; 22 s	15/16 recorded successes; 4.2 agent turns; 16 s

The larger early difference rests on one run per task and predates changes to the harness and execution code. The later comparison recorded equal success counts with fewer turns and less time for direct shortcuts. These results support a bounded efficiency finding for the tested Settings tasks, not a general success advantage. Neither comparison involved participants, so neither measures effects on understanding or control.

3.7.3 Observation, Execution, and Study Apparatus

The observation decision was tested on a Pixel 7 running Android 16. The test captured an Accessibility snapshot immediately before and after an `adb shell uiautomator dump`. It rejected a capture when the action-relevant Accessibility state changed across that interval. At the C1 and C2 checkpoints, the shell hierarchy omitted the blocking overlay that Accessibility retained (Table 3.4). A deliberately changing screen was also rejected as an unstable capture. These findings support using Accessibility as Caddie’s production authority and retaining the shell dump only as an external diagnostic. They cover one device and selected interface states, not every application or Android surface.

Table 3.4 summarizes the asymmetric observation that informed this decision. The entries describe the recorded window and node properties. They do not claim that Accessibility is universally more complete than every UI Automator client.

Table 3.4: Controlled observation comparison on a Pixel 7 with Android 16.

Observation	Accessibility snapshot	UI Automator shell dump
Blocking Caddie window	Retained and visible	Omitted
Covered application target	Retained, marked not visible to the user, and rejected	Exposed as enabled and clickable
Role in Caddie	Production observation authority	External diagnostic only

Automated tests exercise the normal-mode verified-action contract at smaller boundaries. Target resolution rejects missing, ambiguous, hidden, or blocked elements. Interface-changing tools require a postcondition, a durable attempt is written before dispatch, and an uncertain result pauses instead of being repeated. The study runner follows a separate contract. It resolves each predefined step against a fresh Accessibility snapshot and treats an Android action as executed when the platform accepts it or reports that the requested state already holds. It then evaluates task-level verification rules at the end of the trial. The study runner does not use the normal executor's per-action durable attempt journal or per-action postcondition check.

The completed study provides separate operational evidence. The retained P02–P19 archives account for 108 scheduled trials across the 18 analyzed participants. They show that the native apparatus supported each complete six-trial workflow and produced the expected study records; behavioral task outcomes are evaluated separately. The records substantiate operation of the native workflow rather than its performance on the participant tasks.

3.8 Reproducing Caddie and Technical Boundaries

The Caddie source repository is hosted at <https://github.com/cruv3/caddie>. It contains the implementation and retained study apparatus referenced by this thesis. The model-trial evidence described in Section 3.7 is available under `research/evaluation/model-trial-screenshots/`. The directory contains the recovered screenshots, their manifest, and a script written to check all 84 expected run slots, file associations, hashes, and PNG integrity.

The accompanying repository provides the reproduction reference. Setup requires Git LFS assets, JDK 21, Android SDK 36, and an Android 16 device or emulator. The setup guide provides the clone, asset-download, build, and installation commands for the `normalDebug` variant.

Study reproduction additionally requires the `studyDebug` variant, seven resettable task applications, and the separate Training Sandbox. The research guide documents their builds. On Windows, `scripts/install-study-fixtures.ps1` with `-IncludeStudyFixtures` builds the study APK and installs the fixture APKs. The study APK itself must also be installed through Android Studio or ADB. Initial fixture states must be prepared before trials. The operator guide documents session operation, exports, and reset. Its older installation and participant-ID instructions must be read alongside the build distinction here and the final cohort described in Chapter 4.

Caddie's setup screen requires Accessibility access, permission to display over other apps, and microphone access before showing the ready screen (Figure 3.2). Normal mode also requires a reachable model service, configured through the server address and model identifier in Figure 3.6.

In the study build, *Study-Portal* *hosten* starts the phone's web server. The investigator opens its displayed address in a laptop browser through Tailscale. Access to the default

3 Caddie: System Design and Implementation

port 8787 must be restricted as described in the operator guide. Hosting the portal does not activate a trial. After session creation, consent, and practice, preparing the next trial arms its assignment and waits for the participant’s spoken request. The study build, portal hosting, and trial activation are separate prerequisites. Figure 3.7 shows the hosting control and browser view.

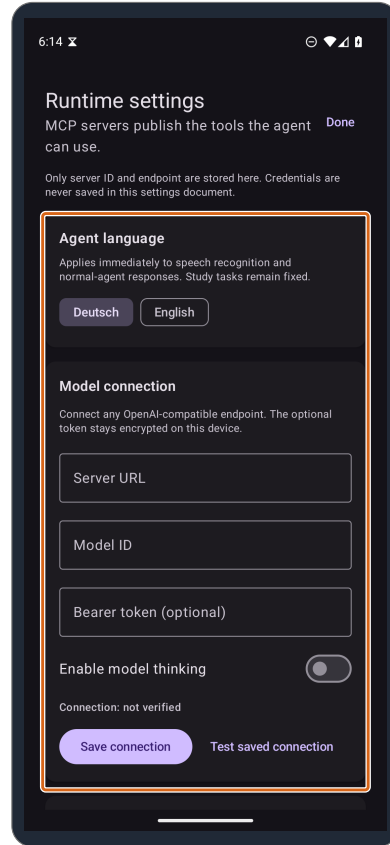


Figure 3.6: Normal-mode connection and language settings. The server URL and model ID specify the model service, with an optional bearer token. The language selector offers German and English.

Normal mode adds a separate model-service dependency. During the unmeasured practice phase, Caddie’s normal mode requested `qwen3.6-35b-a3b` through the self-hosted Model Gateway. The public implementation stores the server address and model identifier in device-local settings so that they can be changed without rebuilding Caddie. For a fixed deployment, Caddie could instead be adapted to read the same values from a local build configuration. Neither option bundles a generative model. Repeating model-dependent runs therefore requires the served model artifact and quantization, gateway configuration, and any optional MCP servers to be stated. The model identifier alone does not fix these inputs. In contrast, the six measured study tasks use predefined specifications rather than model-selected actions.

Normal Caddie use exposes some data beyond the Android process. Each request to the self-hosted model contains the user’s task and the accumulated conversation between

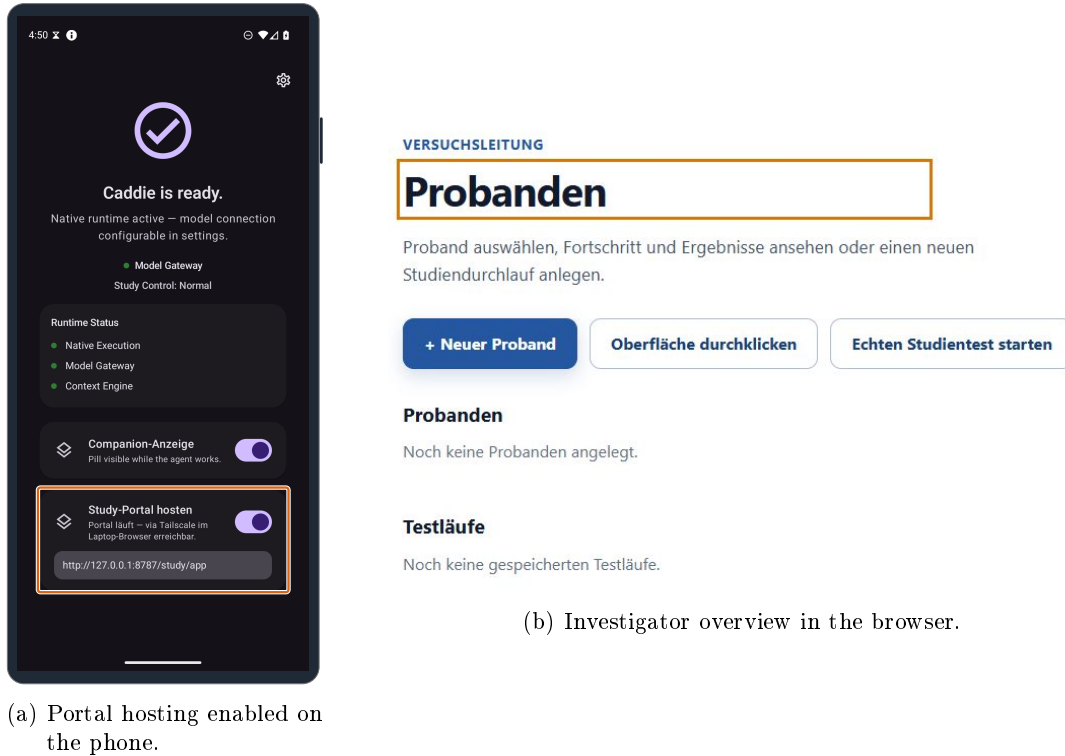


Figure 3.7: Phone-hosted study portal and investigator view in an illustrative setup without participant records. The orange outline on the phone identifies the hosting control. The emulator’s local address was accessed through port forwarding for this capture; study operation used the private network described in the text. The phone frame is schematic; the browser view is cropped to the investigator controls and record lists.

the model and tools. When that conversation includes an interface observation, its structured result can contain visible text, content descriptions, resource identifiers, state flags, and element geometry. Before sending an interface observation, Caddie removes the text and content descriptions from fields that Android identifies as password fields so that these values are not included in the serialized observation sent to the off-phone model service. Other visible application content can still reach the off-phone inference service. A configured MCP server receives the arguments of a remote tool when that tool is invoked, and its returned result can enter a later model request as part of the conversation. Self-hosting and password-field redaction reduce particular disclosures. The privacy boundary also depends on the configured model and MCP endpoints, their network protection, and their logging and retention policies. Caddie cannot be described as generally privacy-preserving.

These constraints bound Caddie’s current capabilities. Normal-mode generative inference depends on a reachable off-phone model service. The retained task trials cannot establish whether the selected model fits a phone’s memory, latency, energy, and thermal limits. Accessibility may expose incomplete or poorly labeled interfaces, and Caddie’s normal-mode risk classifier cannot infer every consequence from tool names

3 Caddie: System Design and Implementation

and visible labels. When an interrupted or uncertain action cannot be verified safely, the normal agent pauses for human review instead of attempting automatic recovery. Replay provides guidance rather than execution, and MCP adds normal-mode tools only when an external server is configured. The Accessibility comparison covers one phone, while automated or ADB-injected gestures cannot establish how consistently physical participant touches trigger the C3 pause across devices.

Chapter 3 shows how Caddie keeps orchestration and tool routing under the control of the Android application. The same runtime displays the oversight controls, identifies current interface targets, and retains selected runtime and task-level study records. In normal mode, the model selects tools and receives retrieved guidance. In study mode, predefined trial specifications replace model-selected actions while the same participant controls remain available. Accessibility, MCP, HTTP, Room, and retrieval are adopted mechanisms rather than individual contributions of Caddie. Caddie contributes their combination and defines explicit limits for verification and intervention.

Chapter 4 specifies how this apparatus was used in the controlled study and how its records were analyzed.

4 Study Design

This chapter describes the controlled within-participant study used to examine how required oversight shapes interaction effort, workload, perceived control, the perceived ability to intervene in time, and participant-reported detection of introduced errors. Each participant encounters all three oversight conditions across tasks of lower and higher criticality. The design supports within-person comparisons while keeping the interpretation bounded to the six study tasks, their predefined error variants, and the implemented apparatus. The chapter also distinguishes the outcomes that address the research questions from secondary and exploratory measures and documents the procedure, analysis plan, and data provenance needed to interpret the results.

4.1 Research Questions and Hypotheses

The study addresses the same two research questions introduced in Chapter 1. RQ1 concerns the effort and experience associated with different levels of required oversight. RQ2 concerns whether participants report detecting a predefined error and the stage at which they report doing so, considered across oversight condition and task criticality.

RQ1 How do different levels of required oversight affect objective interaction effort, subjective workload, perceived control, and the perceived ability to intervene in time when people use an LLM-based smartphone agent?

RQ2 How does participant-reported detection of deliberately introduced errors vary between low- and high-criticality smartphone tasks and among the three levels of required oversight, in terms of detection probability and reported stage?

The two questions separate the interaction costs of oversight from its role in detecting introduced errors. Objective interaction effort refers to required participant decisions and their cumulative response time rather than to the number of actions executed by the system. Detection stage is reported retrospectively after each task. It is not treated as a contemporaneous measure of the participant’s private moment of recognition.

Prior work motivates a trade-off but does not determine the ordering of the three Caddie conditions. Confirming each step can limit error propagation at the cost of repeated interaction. A final checkpoint reduces immediate involvement but can make later diagnosis and recovery more costly [42]. User involvement during both planning and execution has also produced higher cognitive load in a simulated assistant study [17]. Neither study compared C1, C2, and C3 on a smartphone. The RQ1 hypotheses are therefore predictions for this experiment:

- H1a** The number and cumulative response time of required participant decisions are expected to be highest in C1, lower in C2, and lowest in C3.
- H1b** Unweighted Raw NASA-TLX workload differs among the three conditions, with lower workload expected in C2 than in C1 and C3.
- H1c** Perceived control and the perceived ability to intervene in time are expected to be highest in C1, lower in C2, and lowest in C3. The two items are analyzed separately and are not treated as a validated scale.

H1b assumes that one final checkpoint reduces repeated interruptions compared with C1. It also gives participants a defined opportunity to review the action, which may reduce the need to decide when to monitor Caddie compared with C3. These expected benefits motivate the hypothesis of lower workload in C2.

The RQ2 hypotheses concern participant reports that match the controlled error assigned to a trial. They do not treat an unrelated anomaly report as detection and do not infer an unspoken recognition time:

- H2a** Participants are more likely to report detecting the deliberately introduced error in high-criticality than in low-criticality tasks.
- H2b** Detection probability is expected to be highest in C1, lower in C2, and lowest in C3.
- H2c** The difference between high- and low-criticality tasks is expected to be largest in C3 and smallest in C1.
- H2d** Among trials coded as detections, reports of noticing the error before or during execution are expected to be more common in C1 and C2 than in C3. Reports of noticing it only when viewing the result are expected to be more common in C3. This hypothesis is evaluated descriptively because the questionnaire cannot distinguish a C1 gate or C2 checkpoint from another moment during execution.

H2c assumes that higher task consequences encourage closer attention when no confirmation is required. Mandatory checks in C1 may instead prompt review even for lower-criticality tasks, reducing the expected criticality gap.

4.2 Experimental Design

The experiment compares three levels of required participant oversight across tasks predefined as lower or higher in criticality. Both factors vary within participants. The allocation matrix assigns task pairs to oversight conditions, selects three controlled-error trials, and balances the conditions across the three two-trial periods.

4.2.1 Oversight Conditions

The independent variable is the level of required oversight. In all three conditions, Caddie displays the current action and retains the same voluntary controls for pausing, stopping, and correction. C1, termed *stepwise confirmation*, adds a mandatory decision before each configured step that advances the requested task. Interface-only preparation, such as focusing an input field or closing the keyboard, may proceed without a gate. A participant can approve continuation or use the shared intervention controls while the confirmation is pending. C1 and C2 use the same swipe-to-confirm control. They differ in the number and placement of mandatory confirmations, and C2 bundles review information at its final checkpoint.

C2, termed the *final checkpoint*, removes the stepwise gates. Caddie can perform preparatory steps without a required response, but pauses immediately before the commit step that makes the requested change take effect. The checkpoint summarizes the pending actions and relevant values so that the participant can approve the commit or intervene. Each trial specification contains exactly one such commit step, which fixes the location of the C2 gate for that task.

C3, termed *voluntary intervention*, adds no condition-defined mandatory confirmation. Caddie continues to show its current action, and the participant can still pause, stop, or correct the run. Technical failures and unclear corrections are handled independently of the experimental condition: a technical failure ends the trial, whereas an unclear correction keeps it paused. Across C1, C2, and C3, the assigned trial specification determines the nominal actions, controlled-error variant, and final verification rules. The language model does not select the study actions.

4.2.2 Task Criticality and Controlled Errors

Task criticality is a second within-participant factor with two intended levels. The low-criticality tasks ask Caddie to share an estimated arrival time with a contact, transfer two items from a whiteboard photograph into a note, or add a recommended song to a music library. The high-criticality tasks change a calendar appointment, configure Do Not Disturb for an examination, or complete a payment. All three high-criticality outcomes are represented in local study fixtures rather than participants' accounts or the Android system settings. This classification was fixed in the task specifications before a participant session. It reflects the expected severity of an incorrect result in the study scenario rather than a universal ranking of the applications. A seven-point post-task rating asks how severe the possible consequences of an incorrect result would be and serves as a manipulation check for this intended distinction.

Each task specification defines one controlled error by replacing a correct target or value with a predefined incorrect alternative. A variant is enabled only when the participant matrix assigns an error to that trial. The remaining specification and its final verification rules stay fixed. The banking application is a local study fixture, so the task moves neither real money nor personal financial data. An accepted correction before a commit

rewinds the active trial to the relevant step. After a commit, the calendar-appointment, Do Not Disturb, notes, and music tasks provide compensating sequences that restore and verify the specified state. In the conducted study, Maps/Messenger and banking accepted corrections before their commit steps but did not provide post-commit recovery.

Table 4.1 places the six controlled alternatives and their task-specific commit boundaries alongside the fixed pairs of low- and high-criticality tasks.

Table 4.1: Fixed task pairs, controlled alternatives, and commit boundaries.

Pair	Task	Intended → controlled alternative	Commit boundary
1	Maps/Messenger (low)	Campus Deutz → Cologne Central Station	Send message
	Email/Calendar (high)	15:00–16:00 → 16:00–17:00	Save calendar event
2	Gallery/Notes (low)	Thursday → Tuesday	Save note
	Calendar/Do Not Disturb (high)	10:00–11:00 → 12:00–13:00	Save Do Not Disturb rule
3	Chat/Music (low)	Recommended original song → sped-up version	Add song to library
	Email/Banking (high)	EUR 30 → EUR 80	Send simulated payment

Detection of the deliberately introduced error is derived from the standardized questionnaire completed after each task. A report is counted as detection only when the participant’s description identifies the incorrect value, target, or parameter assigned to that trial. Detection and correction are coded separately, so a matching report counts as detection even if the participant took no corrective action. The reported timing categories are before execution, during execution, only when viewing the result, and not precisely determinable. They cannot identify a particular C1 gate or C2 checkpoint. The complete exposure and coding rules are defined in Section 4.6.2.

4.2.3 Within-Participant Allocation and Counterbalancing

Each participant completes all six tasks once. The matrix groups them into three fixed pairs of low- and high-criticality tasks: Maps/Messenger with Email/Calendar, Gallery/Notes with Calendar/Do Not Disturb, and Chat/Music with Email/Banking. One pair is assigned to each oversight condition. Every participant therefore completes two tasks under C1, two under C2, and two under C3, with one low- and one high-criticality task in each condition. Each participant completes the three pairs in three consecutive two-trial periods. Exactly three of the six trials contain an assigned controlled error, one under each oversight condition. The matrix specifies which task within each condition receives that error.

Figure 4.1 summarizes the allocation within one participant and the construction of the complete 18-slot matrix.

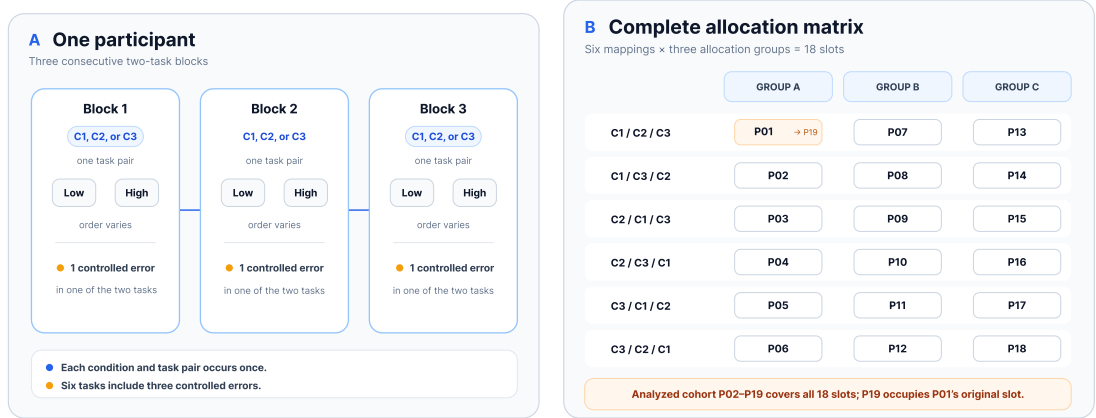


Figure 4.1: Construction of the deterministic participant-allocation matrix. Each participant completes three two-task blocks, with one task pair and one oversight condition per block and one controlled-error trial per condition. Each of the six assignments of C1–C3 to task pairs is represented by three matrix slots. Each slot specifies the task order and controlled-error assignments. The 18 analyzed participants cover every matrix slot exactly once.

The 18 analyzed participants cover all 18 allocation-matrix slots exactly once. Within this cohort, each oversight condition occurs equally often in every period, each Oversight × Criticality cell contains nine assigned-error trials, and every task is selected for a controlled error nine times. The pair combining Maps/Messenger and Email/Calendar is presented low-first and high-first to nine participants each. For each of the other two pairs, 11 participants complete the low-criticality task first and seven complete the high-criticality task first. Task order is therefore not fully balanced. Trial period and task identity remain attached to the analysis records. Assignments are derived from the participant identifier before the session, and the study runtime does not randomly select an error trial.

4.3 Apparatus and Study Materials

The study materials support both task execution and the collection of participant reports. The following subsections describe the Caddie study build, the task applications and their content, and the questionnaires and interview.

4.3.1 Caddie Study Implementation

The accompanying repository provides Caddie's study implementation. Retained device records from P04 onward identify the study phone as SM_G950F. The implementation contains the six task specifications, deterministic runner, instruments, matrix,

portal, and reset procedure. The measured trials use the fixed specifications without asking a language model to select actions. Only the preceding free practice uses normal mode through the off-phone Model Gateway.

The runtime requires Caddie’s Accessibility service, overlay and foreground service permissions, the seven measured fixture applications, and their initial states. The phone serves the study portal to the browser used during the session. Reproduction therefore also requires the device and Android build, fixture versions, permissions, and external Google Maps state to be recorded.

4.3.2 Tasks and Controlled Study Applications

The measured tasks use seven purpose-built fixture applications: Mail, Calendar, Gallery, Notes, Telegram, Music, and Bank. They contain the fixed messages, events, images, songs, and payment data defined by the task specifications. Maps/Messenger obtains a current public-transport duration from Google Maps and sends the resulting message through the Telegram fixture. The Calendar/Do Not Disturb task stores an examination rule in the Calendar fixture rather than changing Android settings. Before the six measured tasks, participants practiced freely in Caddie’s normal mode. This unscripted phase was not analyzed as a trial. Participants therefore used controlled study content instead of personal data, while the Maps route remained dependent on an external application and current network results.

Figure 4.2 shows reconstructed fixture states for the three high-criticality outcomes.

Each specification defines the participant goal, required applications, ordered actions, controlled-error substitution, commit step, and observable postcondition. Verification checks the resulting fixture state rather than only whether a commit occurred. It checks the requested message value, note contents, music selection, meeting time, Do Not Disturb period, or simulated payment recipient, reference, and amount, as applicable.

The portal resets the controlled applications during trial preparation and again after the post-task workflow. The next trial starts only after the fixture content has been restored and the phone has returned to the home screen. This removes saved notes, calendar changes, music-library additions, messages, and simulated payments produced by the preceding trial. Google Maps remains an external application whose route result and starting state must be prepared separately.

4.3.3 Questionnaires and Interview

After each task, participants first report whether anything about the execution or result stood out to them. An affirmative or uncertain response is followed by an optional description, the stage at which it was noticed, and the participant’s reaction. For RQ2, an affirmative answer counts as detection only if its description identifies the assigned incorrect value, target, or parameter. The same questionnaire asks how well the result matched the task goal, how confident the participant is in that assessment, and how

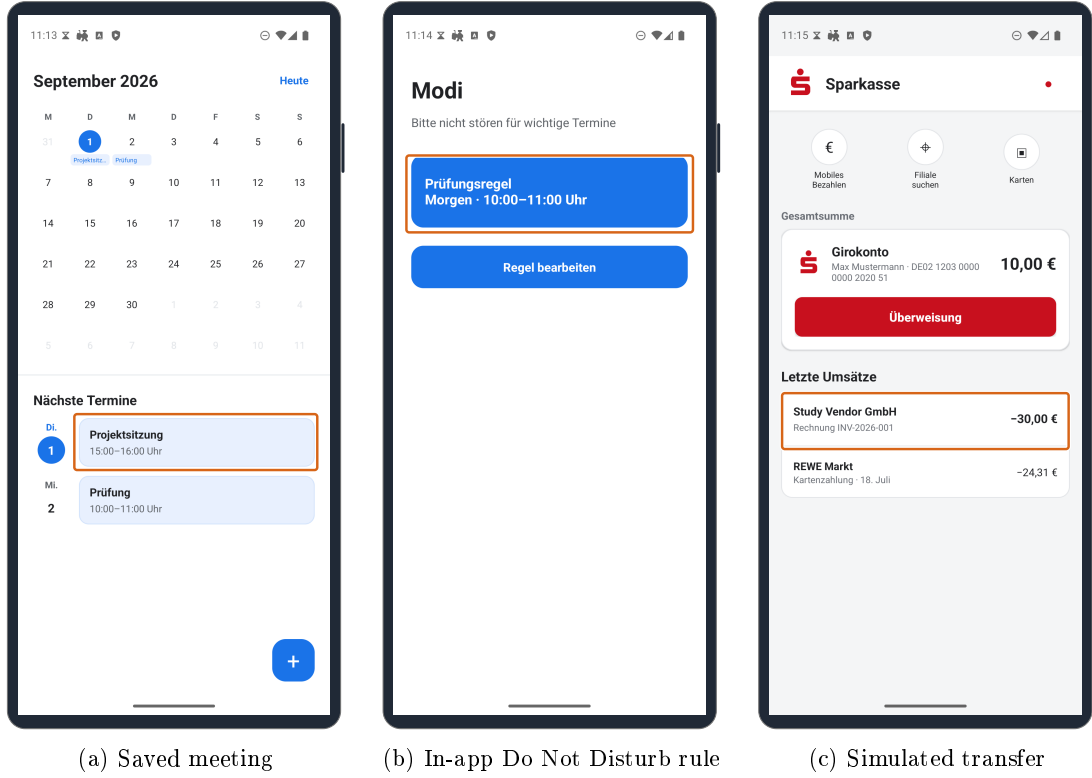


Figure 4.2: Reconstructed apparatus captures of the three high-criticality outcomes using synthetic intended values: a saved meeting in Study Calendar, an examination-related Do Not Disturb rule stored inside Study Calendar, and a completed EUR 30 transfer in Study Bank. The captures were produced from the retained fixture implementation and were not retained from participant sessions. The Do Not Disturb state does not change Android system settings, and the transfer uses no real money or personal account data.

severe the consequences of an incorrect result would be. Result match and confidence are secondary task-level measures. The seven-point consequence rating checks the intended low- versus high-criticality distinction.

After both tasks assigned to one oversight condition, participants complete a condition-level questionnaire. It contains the six NASA-TLX dimensions of mental demand, physical demand, temporal demand, performance, effort, and frustration on 0–100 scales in five-point increments [25]. Their unweighted mean is the Raw NASA-TLX workload measure for RQ1. Two separate seven-point items assess whether participants felt in control and whether they could stop or correct Caddie in time. These two study-specific items are analyzed separately. Additional items cover the appropriateness of confirmation frequency and knowing when attention was needed. Four understandability and predictability items and two trust items were adapted from the Trust in Automation questionnaire [21]. Caddie’s wording and seven-point response format differ from the original instrument, so these six items are retained as exploratory measures rather than treated as the validated complete scale.

The final questionnaire records age, gender, Android experience, and experience with AI, voice, or smartphone assistants. Participants then rank the three oversight conditions and select which form of oversight they would prefer for tasks with lower and higher possible consequences. A six-question semi-structured interview covers the overall experience, differences among the conditions, perceived control, attention strategies, suspicious moments, and the boundary between tasks participants would delegate and tasks for which Caddie should ask first. The questions are presented in the participant-facing portal, with one text field per question. Responses are submitted as written answers before the debrief reveals the assigned errors. They support qualitative interpretation and do not replace the predefined RQ1 and RQ2 outcomes.

4.4 Participants and Data Handling

The planned sample was 18 participants because the counterbalancing matrix contains six possible assignments of C1, C2, and C3 to the three task pairs, each represented by three allocation slots. No statistical calculation was made before the study to estimate how many participants would be needed to detect differences between the oversight conditions. One additional participant completed the study. Recruitment then ended because further sessions were not feasible within the remaining thesis schedule. P01 was excluded before outcome analysis because the separate consent record for this session was not recovered. P19 had received the same condition, task-order, and error assignments as P01, so P02–P19 cover all 18 matrix slots exactly once.

The analysis included data from 18 participants who completed a total of 108 study tasks. Participants were between 22 and 35 years old ($M = 28.5$, $SD = 3.6$). Ten participants identified as male and eight as female.

Convenience sampling drew participants from personal, workplace, and private social networks. The sample included relatives, friends, acquaintances, coworkers, and other people met in private social settings. Participation was unpaid and carried no academic benefit. The study did not define formal thresholds for age or prior technical experience. Sufficient German proficiency to understand the study information, tasks, questionnaires, and interview was the only practical eligibility requirement. All enrolled participants were adults.

Participants were excluded only if they withdrew, consent documentation was missing, they did not complete the six study tasks, or their records could not be linked to their assignment and responses. Only P01 was excluded. Missing or unclear observations for a particular measure were handled within that analysis without excluding the participant.

Before practice or any measured task, the portal presented German study information covering the purpose, procedure, expected duration, recorded data, possible burden, voluntary participation, and the right to stop or request deletion while records remained

linked by participant ID. Participants entered their name and explicitly confirmed understanding, the opportunity to ask questions, voluntary participation, pseudonymized processing, and the right to withdraw. Practice began only after these confirmations.

Controlled errors were not disclosed beforehand because doing so would have directed attention toward error search. The debrief after the questionnaires and interview disclosed the controlled-error variants that had been injected.

The research export identifies participants by ID. Names and digital consent confirmations are stored in a separate consent file linked to the corresponding study session. This link allows the investigator to locate a participant's records for a deletion request. Results are reported for the group without identifying individual participants.

Participants could request deletion by giving the investigator their participant ID while their records remained identifiable. The participant information did not specify a fixed deletion date. Consent records will be kept through data analysis, examination of the thesis, and any resulting corrections, then deleted when no longer needed for those purposes. Research data will only be retained beyond that point after checking that free-text responses and other files contain no reasonably identifying information.

4.5 Procedure

After consent, participants entered a free practice phase in Caddie's normal mode. The portal prepared the Android home screen, and participants could choose any harmless task and try pausing, resuming, correcting, or stopping the agent as needed. There was no predefined practice task. Practice remained outside the six measured trials and did not change the assigned condition, task, or error sequence. Once participants indicated that they were ready, the portal prepared the first trial from their matrix assignment.

At the start of each trial, the participant-facing portal presented a task briefing: a short situation explaining why the action was needed, the applications involved, the intended goal, and any values required for the task. The matrix fixed the task itself, but not the wording of the spoken request. Participants could use the displayed request as written, adapt it, or formulate the task in their own words before addressing Caddie. The deterministic study runner then executed the assigned task under C1, C2, or C3. Participants could use the controls available in that condition while Caddie worked. When the run ended, the portal presented the task questionnaire. Participants first recorded whether anything about the execution or result had stood out to them and then completed the remaining task-level ratings. After submission, the device state was reset before the next trial.

Figure 4.3 shows the participant-facing task briefing and the opening anomaly question of the post-trial questionnaire. Both views reproduce the portal implementation used in the study with synthetic placeholder data.

4 Study Design

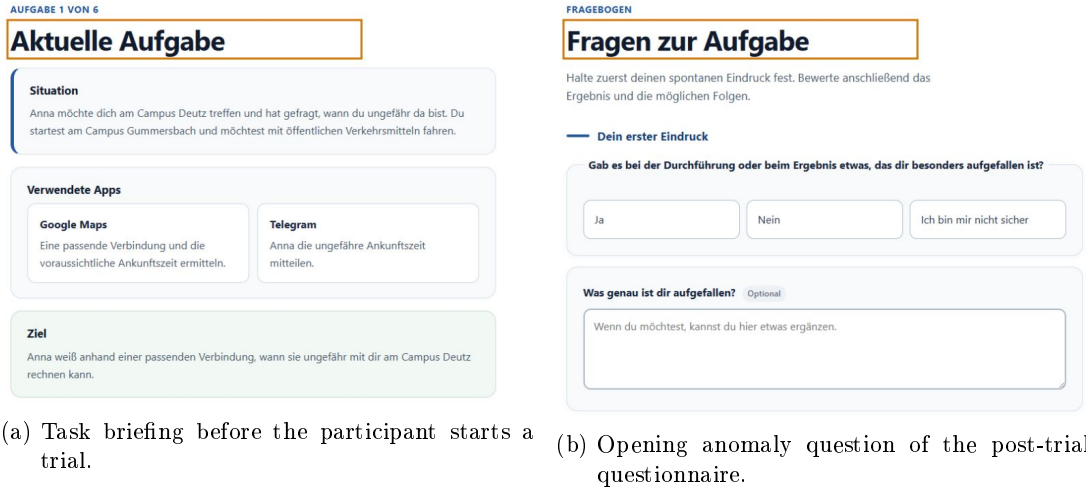


Figure 4.3: Selected views of the German participant interface in the study portal, cropped to the briefing content and the opening anomaly question.

Each condition comprised two consecutive trials. Participants completed the condition-level questionnaire after each pair before moving to the next condition. After the third pair, they provided demographic information, ranked the three conditions, and completed the six-question interview. Only then did the debrief disclose the controlled-error variants that had been injected. The participant confirmed the debrief to complete the session. This ordering kept the task reports, condition ratings, ranking, and interview responses separate from the later disclosure of the controlled errors.

Figure 4.4 summarizes this sequence and shows where the fixed participant matrix enters the conducted procedure.

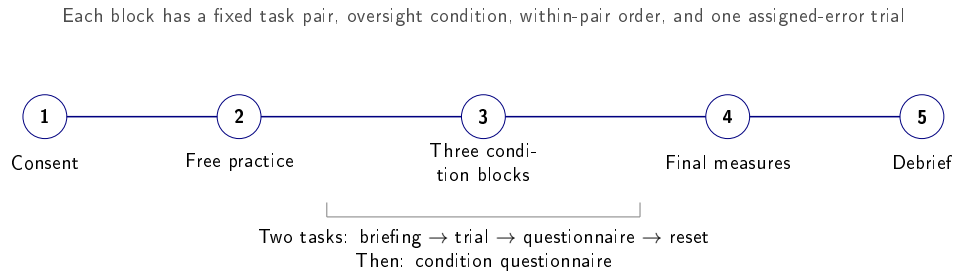


Figure 4.4: Conducted study sequence from consent to debrief. Each condition block contains one low- and one high-criticality task.

4.6 Measures and Operationalization

The measures cover the effort and experience of overseeing Caddie, as well as participant-reported error detection. The following subsections explain how these out-

comes are measured and distinguish the RQ1 and RQ2 measures from secondary and exploratory outcomes.

4.6.1 RQ1: Effort, Workload, and Control

Objective interaction effort is measured for each trial through the number of required participant decisions and their cumulative response time. A required decision is a response to a mandatory C1 stepwise confirmation or the C2 final checkpoint. For a scheduled task with k recorded required decisions, the response times $r_1, \dots, r_k$ are added:

$$T = r_1 + \dots + r_k. \quad (4.1)$$

Each r_j is the time in seconds between presentation of a confirmation and submission of the participant's decision; T is their cumulative response time, not the duration of the complete task. Records from retained attempts of the same scheduled task are combined. C3 has no mandatory confirmation, so both its decision count and T are zero by design. System action count is excluded because it describes the task sequence rather than effort required from the participant. Total trial duration and recorded correction events are retained as secondary interaction measures.

Subjective workload and control are measured once after the two trials in each condition. The Raw NASA-TLX score is the arithmetic mean of the six 0–100 ratings for mental demand, physical demand, temporal demand, performance, effort, and frustration. If these six ratings are denoted by $x_1, \dots, x_6$, the calculation for one participant in one condition is

$$\text{Raw NASA-TLX} = \frac{x_1 + x_2 + x_3 + x_4 + x_5 + x_6}{6}. \quad (4.2)$$

All six ratings receive equal weight, and the resulting score remains on a 0–100 scale. The administered performance scale already ranges from 0 for perfect performance to 100 for complete failure and is therefore not reversed before averaging. Higher Raw NASA-TLX values indicate greater workload. Perceived control and the ability to stop or correct Caddie in time are recorded through two separate seven-point agreement items. They are analyzed individually rather than combined into a scale.

4.6.2 RQ2: Participant-Reported Detection of Introduced Errors

RQ2 uses a yes-or-no participant-reported detection outcome. It includes only trials in which an error was assigned, the participant had an opportunity to encounter it, and the available evidence supports a clear classification. After the task, participants first state whether anything about the execution or result stood out to them and then describe what they noticed. An affirmative response is coded as detection only when this description identifies the assigned incorrect value, target, or task parameter. An intervention or successful correction is not required. Recognition at a C1 or C2 checkpoint also counts when it prevents the incorrect action and no subsequent controlled-error audit event is recorded.

A negative response or an affirmative report that clearly describes a different problem is treated as non-detection only when the technical record shows that the participant had an opportunity to encounter the controlled error. This is established when a controlled-error audit event is followed by completion of the affected step, or when another independent record confirms the opportunity. A trial that ends before the relevant point is classified as unexposed. If the technical records cannot establish whether the error could be encountered, exposure is classified as indeterminate.

A report is also indeterminate when it could refer to the assigned error but does not identify it clearly enough. For example, a general statement that the result was wrong does not specify whether the participant noticed the assigned incorrect time, amount, or other parameter. Such a description could refer to the controlled error or to a different problem, so it is not counted as either detection or non-detection. An “uncertain” answer is treated in the same way. Reports from trials without an assigned error are retained separately as other noticed anomalies or possible false alarms.

All 54 assigned-error trials were reviewed and classified according to the predefined rules. For each trial, the participant’s anomaly answer and description were compared against the task’s predefined incorrect value, target, or parameter. The technical records were also examined to determine whether the participant had an opportunity to encounter the error. Section 6.4 discusses the limitations of this classification procedure.

For affirmative or uncertain responses, participants retrospectively select whether they noticed the anomaly before execution, during execution, only when viewing the result, or could not determine the stage. The reported category is interpreted together with the assigned oversight condition. It is not treated as an independently observed timestamp of the participant’s internal recognition. The final category preserves responses for which the participant could not locate the moment reliably instead of forcing them into a more precise category. Participants also report how they reacted. Runtime intervention, correction, and verification events can corroborate parts of that report but do not replace it. The predefined low- or high-criticality classification of the task is used for the RQ2 comparison. The separate seven-point consequence rating checks whether participants perceived the intended difference and does not redefine the task’s assigned criticality.

4.6.3 Secondary and Exploratory Outcomes

Secondary interaction measures provide context for the primary RQ1 outcomes. Total trial duration and retained explicit-correction events describe task time and recorded control use. The exports lack dedicated participant pause or stop events, so their frequencies cannot be reconstructed. These measures are kept separate because a long trial can reflect execution, device delays, recovery, or participant effort. Task duration excludes apparatus-reset time. Perceived result match and confidence provide task-level context, while the consequence rating checks the intended criticality distinction.

The remaining condition-level items are exploratory. They cover confirmation frequency, knowing when attention was needed, four adapted items on understandability and predictability, and two on trust. Their wording and response scale differ from the Trust in Automation questionnaire, so the six adapted items are analyzed individually rather than as the validated complete instrument. Free-text responses can contextualize unusual ratings but are not an additional confirmatory outcome.

The final ranking of C1, C2, and C3, separate preferences for lower- and higher-consequence tasks, and the six interview responses are exploratory. Rankings and preferences are summarized descriptively. Qualitative content analysis of the written responses starts with categories corresponding to the six interview topics. Recurring points in the responses inform more specific codes. The codebook specifies these codes and their assignment rules before comparisons across conditions. Each assignment is reviewed manually before aggregation, and one response can receive several codes. Results report aggregated themes and descriptive frequencies without individual quotations. Frequencies describe these responses, not prevalence in a wider population. This analysis does not claim inter-coder reliability or introduce a third research question.

4.7 Analysis Plan

The RQ1 analysis approach was documented on 18 August 2026, the RQ2 coding rules on 21 August, and the remaining analysis-plan decisions on 22 August, before the retained statistical results were committed. These decisions were documented during and after data collection. This chronology is not evidence that outcomes had remained unseen when the decisions were made.

The analysis compares C1, C2, and C3 in terms of interaction effort, workload, control, and reported error detection. This section explains how these comparisons are made; Chapter 5 presents their results. Before the comparisons, the data are checked for completeness and the distribution of each measure is inspected. Every participant experienced all three conditions. The models therefore account for repeated observations from the same person, rather than treating them as observations from different participants.

Descriptive summaries report the arithmetic mean (M), median, sample standard deviation (SD), and lower and upper quartiles. The mean is the average, and the median is the middle ordered value, or the mean of the two central values for an even number of observations. The SD describes variation around the mean in the measure's units, not uncertainty about the estimated mean. The quartiles bound the middle half of observations, and their difference is the interquartile range (IQR). They are the 25th and 75th percentiles. Chapter 5 uses brackets to report the two quartile values, not their difference.

Statistical tests use p -values to assess compatibility with an assumption of no difference. A p -value is the probability of a test statistic at least as extreme as the observed one under that assumption and the model assumptions. It is neither the probability that

4 Study Design

a hypothesis is true nor a measure of effect size or practical importance [36]. Tests are two-sided, considering differences in either direction, with a 0.05 threshold. When planned comparisons address the same outcome, the Holm procedure adjusts their p -values to control the risk of at least one false-positive finding in that family [18]. Conclusions are not based on the threshold alone.

Each estimated effect reports its size, direction, and uncertainty. For non-Bayesian models, a 95% confidence interval (CI) gives lower and upper limits for the effect estimate, not the observed-score range. Under the model assumptions, the interval method is designed to include the true effect in 95% of repeated studies. It does not give a hypothesis a 95% probability of being correct [15]. The Bayesian alternative uses credible intervals. Exploratory analyses are identified separately and do not confirm hypotheses after the results are known.

The consequence ratings first check whether participants perceived the high-criticality tasks as more consequential than the low-criticality tasks. For each participant i , the three high-criticality ratings are averaged, as are the three low-criticality ratings. The difference is

$$d_i = \bar{x}_{i,\text{high}} - \bar{x}_{i,\text{low}}. \quad (4.3)$$

Here, each $\bar{x}$ denotes that participant's mean rating at the indicated criticality level. A positive d_i means that the participant rated the high-criticality tasks as more consequential on average. These differences are then summarized across participants. They describe the numerical rating gap; the inferential analysis below instead treats ratings as ordered categories.

An ordinal mixed-effects model estimates the criticality difference while accounting for task position. It treats ratings as ordered categories without assuming equal distances between scale points [11]. Criticality and trial position are fixed effects that estimate systematic differences associated with those factors. A participant-specific random intercept accounts for repeated ratings and general rating tendency. As each task has one criticality level, the check cannot separate criticality from all other task-content differences.

The ordinal models express effects as odds ratios. For outcome probability π , the odds are $\pi/(1 - \pi)$, and the ratio for group A relative to B is

$$\text{OR}_{A:B} = \frac{\pi_A/(1 - \pi_A)}{\pi_B/(1 - \pi_B)}. \quad (4.4)$$

For the ordinal models, the outcome is a rating above a scale threshold. The proportional-odds assumption makes the ratio the same across thresholds while holding other predictors constant [11]. A ratio above one means higher odds in A, below one lower odds, and one equal odds. It does not multiply either rating or probability by that ratio.

Likelihood-ratio tests compare models with and without a focal effect to ask whether allowing it improves how well the model accounts for the data [11]. For criticality, both models retain trial position and the participant random intercept, while the comparison model removes only criticality. The result is reported as $\chi^2(df)$, with one

additional parameter for the high-versus-low comparison. Its p -value uses a chi-square approximation under the no-difference model. The odds ratio and its interval instead describe estimated size and uncertainty. A profile-likelihood interval varies the focal effect and refits the remaining parameters. Its limits mark the likelihood cutoff for a 95% interval [11].

4.7.1 Interaction Effort, Workload, and Control

For H1a, the analysis describes how many required decisions participants made and how much time they spent responding to confirmation requests. These measures are summarized by condition, both for individual trials and for each participant. A participant's condition total is the sum of their two task values, calculated before summarizing these totals across participants. Paired differences compare the same participant's measures across conditions. The counts partly follow directly from the design: C1 requires several decisions, C2 requires one final decision, and C3 requires none. C3's zero is therefore not treated as a statistical discovery. The measures use confirmation responses recorded during trial execution for all 18 participants in the analysis dataset (P02–P19). Task identity, trial position, and error assignment are retained to help interpret differences; these descriptive summaries do not isolate their effects from those of oversight.

For H1b, the analysis asks whether workload is lower in C2 than in C1 and C3. A linear mixed-effects model compares the condition-level Raw NASA-TLX scores. It includes condition and block position as fixed effects, and a participant-specific random intercept. Block position means whether a condition was experienced first, second, or third. The model therefore accounts for order and for differences in participants' general rating levels. The overall test asks whether workload differs anywhere among the three conditions. The two planned comparisons then address the specific prediction: C2 versus C1 and C2 versus C3. An overall difference alone does not establish either of these pairwise differences.

For H1c, perceived control and the perceived ability to stop or correct the agent in time are analyzed separately. Each seven-point item is examined with a cumulative-link mixed model, an ordinal model of the type described above. Both models include condition, block position, and a participant-specific random intercept. Comparisons of C1 with C2 and C2 with C3 assess whether ratings are highest in C1, lower in C2, and lowest in C3, as expected by H1c.

Model checks assess whether these comparisons can be interpreted reliably. For Raw NASA-TLX, residuals are the differences between observed and model-predicted scores. The Shapiro–Wilk W statistic diagnoses departures from the normal error distribution, not workload differences [32].

For ordinal models, proportional odds means the same relative effect at different lower-versus-higher rating thresholds [11]. The diagnostic compares marginal cumulative odds ratios, using a 0.5 continuity correction and thresholds with at least five observations on each side across the compared conditions. It does not adjust for participant or block position and cannot establish the conditional proportional-odds assumption.

If a model is unstable or its assumptions are substantially violated, additional rank-based comparisons are reported. The Friedman test compares each participant’s ranked condition scores and tests for an overall difference, not which pairs differ [13].

The planned pairwise comparisons use Wilcoxon signed-rank tests, which rank the absolute paired differences and retain their signs [38]. Their p -values receive the Holm adjustment. These sensitivity analyses preserve within-participant pairing but do not adjust for block position as the mixed models do. They are an additional check, not a guarantee that all assumptions hold.

A supplementary analysis added during manuscript review describes the median paired differences in perceived control. Conservative 95% confidence intervals use ordered participant differences and binomial coverage [26]. With 18 participants, the fifth and fourteenth ordered differences provide at least 96.9% coverage under independent sampling. These intervals are not adjusted for multiple comparisons or block position and do not invert the signed-rank tests.

4.7.2 Reported Error Detection and Its Timing

The primary RQ2 analysis includes assigned-error trials in which participants had an opportunity to encounter the error and detection could be classified. These are the exposed, determinate trials defined in Section 4.6.2. All qualifying trials from P02–P19 are included. Within each group, the observed detection rate is

$$\text{Detection rate} = \frac{D}{D + N} \times 100\%, \quad (4.5)$$

where D is the number of detected trials and N the number of not-detected trials. Unexposed and indeterminate trials contribute to neither count.

A mixed-effects logistic regression estimates the probability of a trial being classified as detected rather than not detected. It includes condition, task criticality, and trial position as fixed effects, along with a participant-specific random intercept. It also includes an interaction between condition and criticality: this asks whether the high-versus-low criticality difference changes across conditions. A sensitivity model additionally includes task pair to check whether differences among the three paired scenarios substantially change the estimates.

H2a compares high- and low-criticality tasks, averaging the model estimates across oversight conditions. H2b compares all three condition pairs: C1 with C2, C1 with C3, and C2 with C3. The predicted ordering is a higher detection probability in C1 than in C2, and in C2 than in C3. The pairwise p -values receive Holm adjustment. The comparisons report model-based average detection probabilities and odds ratios, with 95% confidence intervals. Unlike the observed detection rate, these probabilities account for the model’s predictors and variation between participants. Reference categories are C2, low criticality, and the first trial position, with the remaining positions represented by categorical indicators. Predictions are averaged equally over the six trial positions and, for H2a, the three conditions or, for H2b, the two criticality levels. The

4 Study Design

resulting average probabilities are used in Equation 4.4, with detection as the outcome and non-detection as its alternative. These are marginal odds ratios: ratios of the odds formed from the averaged probabilities.

H2c asks whether the high-versus-low criticality gap is larger in C3 than in C1. For each condition C , let $\pi_{C,\text{high}}$ and $\pi_{C,\text{low}}$ be the model-estimated average detection probabilities for high- and low-criticality tasks. The calculation has two steps:

$$\begin{aligned}\Delta_C &= \pi_{C,\text{high}} - \pi_{C,\text{low}}, \\ \text{H2c contrast} &= \Delta_{C3} - \Delta_{C1}.\end{aligned}\tag{4.6}$$

First, Δ_C measures the high-minus-low gap within one condition. Then the H2c contrast subtracts the C1 gap from the C3 gap; a positive value is expected by H2c. A probability difference of .10 means ten percentage points, not a relative increase of 10%. A significant difference in one condition and a non-significant difference in another would not, by itself, establish that the gaps differ. With 18 participants and 54 assigned-error trials before exclusions, the comparison is expected to have limited precision. Exclusions can further reduce the number of observations for each combination of condition and criticality. The C2 gap and the other comparisons between gaps are exploratory, with Holm adjustment when p -values are tested.

Unexposed trials are excluded rather than counted as non-detections. Indeterminate trials are also excluded from the primary analysis because their detection status cannot be established. Two sensitivity analyses then examine opposite assignments for these uncertain cases: all are counted as non-detections in one analysis and as detections in the other. This shows how the findings depend on their classification without claiming to know which classification is correct.

The initial logistic model uses maximum-likelihood estimation, which finds the parameter values that best account for the observed data under the model. This approach may become unreliable if some combinations of predictors perfectly separate detections from non-detections, if the fitting procedure fails to reach a stable solution, or if the estimated variation between participants collapses to zero. These problems are called separation, non-convergence, and a singular random-effects fit. If any occurs, the same model structure is estimated using Bayesian multilevel logistic regression.

The Bayesian model combines data with prior distributions, which state assumptions about parameters before observing the data. Fixed effects on the log-odds scale, the natural logarithm of the odds, use normal priors with mean 0 and standard deviation 2.5. The participant-intercept standard deviation uses a half-normal prior with scale 1. Posterior medians and 95% credible intervals replace unstable maximum-likelihood estimates. The posterior is the parameter distribution after combining data and priors. Conditional on the stated model, a credible interval contains 95% of that distribution [15]. These summaries do not use p -values or Holm adjustment.

The computation approximates the posterior with several chains, which are separate sampling sequences. Each discards a warm-up phase used to adjust sampling settings, then retains draws, sampled possible parameter values, for the reported results [28]. Probabilities, odds ratios, and differences are calculated for each draw before reporting

their medians and intervals. Each 95% credible interval spans the 2.5th to 97.5th percentiles of the derived values. A reported median difference therefore need not equal the difference between separately reported median probabilities. The posterior probability that a difference is positive is the share of draws above zero.

For each posterior draw, population-average probabilities account for variation between participants. They are calculated by integrating over the modelled normal distribution of participant intercepts using 30-point Gauss–Hermite quadrature. They neither fix the intercept to zero nor average the fitted offsets of the sampled participants. The same equal weighting of trial positions, conditions, and criticality levels is used as above. The task-pair sensitivity additionally weights the three task pairs equally.

Sampling diagnostics assess whether chains explore the posterior consistently. $\hat{R}$ compares within- and between-chain variation, with values near one indicating agreement. Successive draws are dependent, so effective sample size estimates the equivalent number of independent draws, not the participant count. Bulk and tail values concern central and outer distribution summaries. Divergences flag numerical exploration difficulties [34]. These checks assess computation, not whether model assumptions are correct.

H2d examines when participants reported noticing an error. Among trials classified as detections, counts and proportions summarize reports before execution, during execution, and only when viewing the result. Reports marked “not precisely determinable” are listed separately. Summaries are grouped by condition and criticality. The combined share of before- and during-execution reports in C1 and C2 is compared with C3; the result-only share in C3 is compared with C1 and C2. These are descriptions of recalled timing, not measurements of the exact moment of detection. Since only detected trials contribute and some groups may be small, no additional inferential model is fitted for timing. Reports from trials without an assigned error are summarized separately as other noticed anomalies or possible false alarms. Recorded interventions and corrections help interpret the reports but do not replace the predefined detection rule.

4.8 Study Replication

Before participant sessions, the complete six-task workflow was exercised with synthetic fixture data on the study phone. This rehearsal established that the complete workflow could reach the debrief and produce the expected study records. It is not part of the participant dataset.

The source package submitted with the thesis contains the six task specifications, participant matrix, questionnaire definitions, deterministic runner, portal workflow, reset procedure, and export format. The analysis records retain the matrix slot, trial period, task identity, oversight condition, and controlled-error assignment.

Replicating the study requires the task specifications, the complete allocation matrix, study instruments, RQ2 codebook, reset procedure, and analysis rules described in this

chapter. Section 3.8 explains how to build and install Caddie and its fixture applications, while the operator guide covers readiness checks, live-session operation, export, and reset. Normal-mode practice additionally requires the project-controlled Model Gateway, whereas the six measured trials use deterministic task specifications. Accessibility permissions, network connectivity, and the live Google Maps surface remain environment-dependent parts of a replication.

5 Results

This chapter reports the study outcomes for the final analysis cohort. Results are organized by their role in the analysis: data quality and the manipulation check precede the two research questions, while secondary and exploratory outcomes provide descriptive context. Broader interpretation and implications are reserved for Chapter 6.

5.1 Participant Flow and Data Quality

Participant flow and record completeness establish the dataset used for all subsequent analyses.

Data collection ended after 19 participants had completed the study. One participant's completed session was excluded before outcome analysis because the corresponding consent record, stored separately from the research data, could not be recovered. No other participant met the exclusion criteria described in Chapter 4. The final sample therefore comprised 18 participants and covered each planned combination of condition allocation, task order, and controlled-error assignment once.

The included data contained all planned questionnaire and interview records. All six tasks per participant were completed, giving 108 completed tasks and 108 corresponding task questionnaires. Every participant also completed one questionnaire after each of the three conditions, both parts of the final questionnaire, and six written interview responses.

5.2 Task-Criticality Manipulation Check

The manipulation check tests whether participants rated the predefined high-criticality tasks as more consequential than the low-criticality tasks.

Low-criticality tasks had a median consequence rating of 2 [IQR 2–4], whereas high-criticality tasks had a median of 7 [IQR 7–7]. Table 5.1 counts ratings, not participants, at each point of the seven-point scale, where higher values indicate more severe consequences of an incorrect result. Each row contains 54 ratings because all 18 participants rated three tasks at each criticality level.

For each participant, the three low-criticality ratings were averaged and subtracted from the average of their three high-criticality ratings, giving the difference d_i defined in Equation 4.3. All 18 differences were positive, with a mean of 3.39 scale points.

This is the mean of the within-participant differences, not the difference between the two group medians of 7 and 2. The differences varied across participants ($SD = 1.16$, range 1.67–5.67).

Table 5.1: Consequence-rating distribution by predefined task criticality. Values are counts across 54 ratings per level.

Task criticality	Consequence rating						
	1	2	3	4	5	6	7
Low	13	15	8	7	1	3	7
High	0	1	0	4	4	3	42

The ordinal mixed-effects model estimated higher odds of a higher rating for the predefined high-criticality tasks than for the low-criticality tasks (proportional odds ratio 82.15, profile-likelihood 95% CI [23.29, 351.20]). The interval describes uncertainty about the odds ratio, not the range of participants’ ratings. Its width indicates that the size of the association is imprecisely estimated. Comparing this model with the same model without a criticality effect gave likelihood-ratio $\chi^2(1) = 68.31$, $p < .001$. The result supports the intended distinction between the selected task sets. It does not separate criticality from other differences in task content because each task belonged to only one predefined criticality level.

5.3 RQ1: Interaction Effort, Workload, and Control

The RQ1 analyses compare required interaction, subjective workload, perceived control, and timely intervention across the three oversight conditions. C1 required stepwise confirmation, C2 one final checkpoint, and C3 no mandatory confirmation.

5.3.1 Required Decisions and Response Time

Required-decision counts and cumulative response time describe the required interaction.

Across the 36 scheduled tasks in each condition, C1 required a median of seven participant decisions per task, compared with exactly one in C2 and none in C3. Cumulative response time is the sum of response latencies for all required decisions in one scheduled task. Its median was 13.34 seconds in C1, 5.58 seconds in C2, and zero in C3. Table 5.2 also reports condition totals per participant, each of which combines that participant’s two tasks in the condition.

At both task and participant levels, the median required-decision count and cumulative response time were highest in C1, lower in C2, and lowest in C3, as predicted. For decision count, this ordering largely reflects the condition design.

Table 5.2: Required decisions and cumulative response time by oversight condition. Values are medians with interquartile ranges in brackets.

Condition	Per task ($n = 36$)		Per participant ($n = 18$)	
	Decisions	Time (s)	Decisions	Time (s)
C1	7 [6, 8]	13.34 [10.32, 15.51]	14 [13, 16]	27.22 [25.18, 29.95]
C2	1 [1, 1]	5.58 [4.80, 6.68]	2 [2, 2]	11.40 [10.06, 12.51]
C3	0 [0, 0]	0.00 [0.00, 0.00]	0 [0, 0]	0.00 [0.00, 0.00]

5.3.2 Subjective Workload

Subjective workload was measured with the unweighted Raw NASA-TLX. Each participant's condition score averages the six workload ratings (Equation 4.2).

Raw NASA-TLX was descriptively lowest in C2, as predicted by H1b (Table 5.3). Fourteen of the 18 participants reported lower Raw NASA-TLX in C2 than in C1, and the same number reported lower workload in C2 than in C3.

Table 5.3: Unweighted Raw NASA-TLX workload by oversight condition. Higher values indicate greater workload. There are 18 observations per condition.

Condition	Mean (SD)	Median [IQR]
C1	21.25 (12.88)	26.25 [8.33, 29.17]
C2	16.85 (13.81)	19.17 [4.58, 22.29]
C3	19.81 (12.54)	24.58 [9.38, 27.29]

The linear mixed-effects model did not show an overall workload difference among conditions ($\chi^2(2) = 1.67$, $p = .433$). It estimated workload in C2 to be 4.40 Raw NASA-TLX points lower than in C1 (95% CI $[-11.19, 2.40]$, Holm-adjusted $p = .409$) and 2.96 points lower than in C3 (95% CI $[-9.76, 3.83]$, Holm-adjusted $p = .409$). After Holm adjustment for both planned comparisons, neither comparison was statistically significant.

Because the Shapiro–Wilk diagnostic indicated non-normal model residuals ($W = .915$, $p < .001$), the prespecified rank-based sensitivity analysis was also used. Its Friedman test indicated an overall difference ($\chi^2(2) = 9.03$, $p = .011$), but paired Wilcoxon tests did not establish either planned difference after Holm adjustment. The median paired difference was -6.25 for C2 minus C1 and -4.58 for C2 minus C3 (Holm-adjusted $p = .141$ for both). Thus, workload was descriptively lowest in C2, but the planned comparisons did not support H1b.

5.3.3 Perceived Control and Timely Intervention

Perceived control was examined through separate ratings of perceived control and the ability to stop or correct Caddie in time. Median perceived control was highest in C1,

5 Results

lower in C2, and lowest in C3, as predicted. The median timely-intervention rating was equal in C1 and C2 and lower in C3 (Table 5.4).

Table 5.4: Perceived control and timely intervention by oversight condition. Values are medians with interquartile ranges in brackets. There are 18 observations per condition.

Condition	Perceived control	Timely intervention
C1	7 [6, 7]	6 [6, 7]
C2	6 [5.25, 6]	6 [5, 7]
C3	4 [3.25, 5]	5 [4, 6]

Figure 5.1 places workload, perceived control, and timely intervention side by side and shows the repeated observations underlying the aggregate summaries.

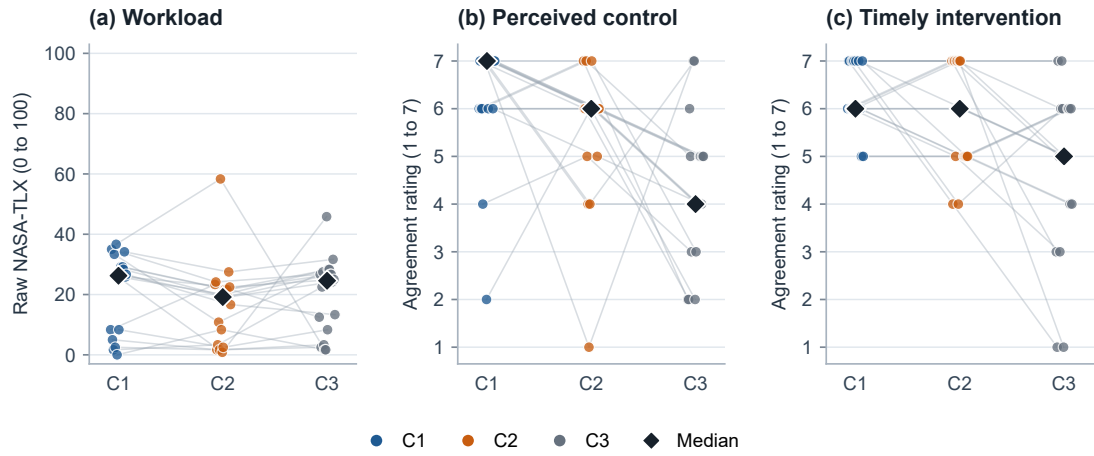


Figure 5.1: Participant-level subjective outcomes across the three oversight conditions (C1, C2, and C3). Thin lines connect each participant's three condition-level observations, point color distinguishes conditions, and diamonds mark condition medians. Higher values indicate greater workload in panel (a), stronger perceived control in panel (b), and a stronger perceived ability to intervene in time in panel (c). Each panel contains 18 participants and 18 observations per condition.

For perceived control, the mixed model estimated almost no differences in participants' general rating levels after accounting for condition and block position. This was treated as a diagnostic warning, so the model's tests and confidence intervals were not used to assess condition differences. The prespecified rank-based tests were used instead. The Friedman test indicated an overall condition difference ($\chi^2(2) = 18.42, p < .001$). However, neither adjacent planned comparison remained significant after Holm adjustment (Holm-adjusted $p = .096$ for C1 versus C2 and $p = .065$ for C2 versus C3).

The supplementary median paired differences were 1 point for C1 minus C2 (conservative 95% CI [0, 1]) and 1.5 points for C2 minus C3 (CI [1, 2]). These unadjusted intervals describe median changes and do not replace the Holm-adjusted signed-rank comparisons.

The model for timely intervention converged away from parameter bounds and was retained. Ratings were more likely to be higher in C2 than in C3 (proportional odds ratio 5.48, profile-likelihood 95% CI [1.44, 23.38], Holm-adjusted $p = .024$), while the difference between C1 and C2 remained unclear (odds ratio 2.98, 95% CI [0.79, 12.16], Holm-adjusted $p = .106$). H1c was therefore partially supported: higher timely-intervention ratings in C2 than in C3 were supported by this model, but the complete predicted ordering was not established for either item.

At informative thresholds, the descriptive cumulative odds ratios all pointed toward higher ratings in C1 than C2 and in C2 than C3, but their magnitudes varied. This does not establish proportional odds. The paired-rank sensitivity gave a Holm-adjusted $p = .056$ for C2 versus C3, so the evidence for this difference depends on the analysis method.

5.4 RQ2: Participant-Reported Detection of Introduced Errors

The RQ2 analyses compare participants' retrospective reports of detecting introduced errors across task-criticality levels and oversight conditions. They also examine whether the difference between criticality levels varied by condition and when participants said they noticed the error. The results use the classifications obtained from the review of all 54 assigned-error trials, applying the rules described in Section 4.6.2.

5.4.1 Detection by Task Criticality

Detection by task criticality was evaluated among the 54 trials with an assigned controlled error. A matching report was classified as detected, a non-matching report with confirmed exposure as not detected, and uncertain report evidence as indeterminate. Thirty-five reports were classified as detected, eight as not detected, and 11 as indeterminate. System logs showed that participants reached the point at which the assigned error could be encountered in every trial. Table 5.5 reports these classifications by predefined task criticality.

All 11 indeterminate cases contained an affirmative anomaly report and a nonempty description, with technical exposure established. Their descriptions did not identify the assigned incorrect value, target, or parameter clearly enough to distinguish recognition of the controlled error from another problem.

Table 5.5: Participant-reported detection of introduced errors by predefined task criticality. The detection rate excludes indeterminate trials.

Criticality	Detected	Not detected	Indeterminate	Detection rate
Low	15	7	5	68.2%
High	20	1	6	95.2%

The observed rates use only detected and not-detected trials (Equation 4.5): $15/(15 + 7) \times 100\% \approx 68.2\%$ for low-criticality tasks and $20/(20 + 1) \times 100\% \approx 95.2\%$ for high-criticality tasks. These are descriptive proportions, distinct from the model-adjusted probabilities below.

In the high-criticality trials of C2 and C3, all six determinate cases in each condition were classified as detections and none as non-detections. This perfect split prevented the planned frequentist model from estimating stable coefficients, a problem known as complete separation. The prespecified Bayesian model was therefore used. It estimated an average detection probability of .91 for high-criticality tasks and .71 for low-criticality tasks. Calculating the high-minus-low difference within each posterior draw gave a median difference of .19, or 19 percentage points (95% credible interval [.01, .39]), with a marginal odds ratio of 4.38 (95% credible interval [1.03, 34.92]) and a posterior probability of .979 that the high-criticality probability was greater. Under the primary specification, the result supported H2a.

All sensitivity analyses retained the direction of this difference, but they differed in how certain it was. The task-pair model produced the same rounded probability difference, but its interval included zero (95% credible interval [−.003, .376]). The interval also included zero when all indeterminate trials were coded as non-detections ([−.08, .35]). It excluded zero when all indeterminate trials were coded as detections ([.01, .34]). Thus, reported detection was consistently estimated as higher for high-criticality tasks, but the strength of the evidence depended on the model and the treatment of indeterminate trials.

The four Bayesian models showed no reported sampling warnings. Each used four chains with 3,000 warm-up iterations and 4,000 retained draws per chain. No divergent transitions occurred, the maximum $\hat{R}$ was 1.00 to the reported precision, and the smallest bulk or tail effective sample size was 4,876. These checks support numerical estimation, but do not establish that the model assumptions are correct or that the study hypotheses hold.

5.4.2 Detection by Oversight Condition

Table 5.6 compares the observed classifications and determinate detection rates for C1, C2, and C3, averaged across task criticality.

Table 5.6: Participant-reported detection of introduced errors by oversight condition. The detection rate excludes indeterminate trials.

Condition	Detected	Not detected	Indeterminate	Detection rate
C1	13	4	1	76.5%
C2	11	1	6	91.7%
C3	11	3	4	78.6%

5 Results

Figure 5.2 displays the estimates for low- and high-criticality tasks within C1, C2, and C3. It shows the 95% credible intervals and the excluded indeterminate count for each combination.

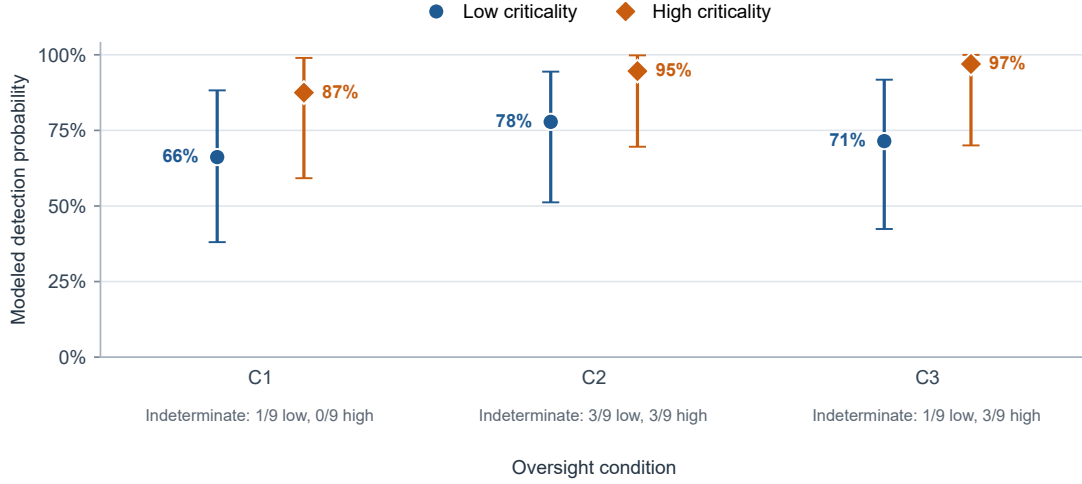


Figure 5.2: Model-estimated probabilities that participants reported detecting a deliberately introduced error, shown separately for low- and high-criticality tasks in C1, C2, and C3. Points and whiskers show posterior medians and 95% credible intervals from the primary Bayesian mixed-effects logistic model. The model used 43 trials classified as detected or not detected from 18 participants. The annotations report the excluded indeterminate count for each condition-criticality combination, totaling 11.

The criticality-averaged estimates from the primary Bayesian model did not follow the predicted pattern of detection probability being highest in C1, lower in C2, and lowest in C3. The estimated detection probability for C1 was .09 lower than for C2 (95% credible interval $[-.30, .13]$) and .07 lower than for C3 (95% credible interval $[-.29, .16]$). C2 was estimated .02 higher than C3 (95% credible interval $[-.19, .23]$). All three pairwise intervals included zero, so the data did not establish differences in detection probability among the oversight conditions. The task-pair model and both indeterminate-trial bounds likewise produced intervals that included zero for every condition contrast, while the point ordering changed across these specifications. H2b was therefore not supported.

5.4.3 Criticality Difference Across Oversight Conditions

This analysis examined whether the high-minus-low detection difference varied across oversight conditions, as predicted by H2c.

The estimated high-minus-low detection-probability difference was .20 in C1 (95% credible interval $[-.11, .50]$), .15 in C2 (95% credible interval $[-.05, .38]$), and .23 in C3 (95% credible interval $[-.03, .52]$). The C3 estimate was the largest, as predicted, but the C2 estimate rather than the C1 estimate was the smallest. More importantly, the high-minus-low difference in C3 exceeded the corresponding difference in C1 by only

.03, with a wide credible interval that included both negative and positive values (95% credible interval $[-.32, .42]$). The posterior probability that this C3-minus-C1 difference was positive was .571. The task-pair model produced a similar estimate of .03 (95% credible interval $[-.33, .38]$), while the two analyses that recoded indeterminate trials gave estimates with opposite signs. The predicted condition-by-criticality pattern was therefore not established, and H2c was not supported.

5.4.4 Reported Detection Stage

Reported detection stage was used to assess H2d. Among the 35 detected trials, participants reported noticing the error during execution in 23 trials and only when viewing the result in 12. No participant selected the option that the stage could not be determined precisely. Table 5.7 separates these counts by oversight condition and task criticality.

Table 5.7: Participant-reported detection stage among trials classified as detections. Values are counts; all 35 detected trials are included.

Condition	Criticality	During	Result only	Not precise
C1	Low	4	1	0
C1	High	7	1	0
C2	Low	2	3	0
C2	High	2	4	0
C3	Low	4	1	0
C3	High	4	2	0

For the comparison in H2d, 15 of 24 detections (62.5%) in C1 and C2 combined were reported during execution. The corresponding count in C3 was 8 of 11 (72.7%). Result-only reports accounted for 9 of 24 detections (37.5%) in C1 and C2 combined and 3 of 11 (27.3%) in C3. Both comparisons ran counter to the predicted direction. C1 alone had the largest during-execution share (11 of 13, 84.6%), whereas C2 had the largest result-only share (7 of 11, 63.6%). The descriptive expectation in H2d was therefore not supported.

5.4.5 Reported Anomalies Without an Assigned Error

An anomaly was reported in six of the 54 task questionnaires from trials without an assigned controlled error; the other 48 reports were negative. These responses were retained separately from the RQ2 classifications. Without an assigned controlled error or an adjudication of the reported content, an affirmative response does not establish whether it reflects another anomaly or a false alarm.

Hypothesis Overview

The confirmatory results were mixed. H1a was consistent with the planned ordering of required decision count and cumulative response time, although this ordering partly follows from the gate design. H1b was not supported: workload was descriptively lowest in C2, but the planned contrasts did not establish either predicted difference. H1c was partially supported because timely-intervention ratings supported C2 over C3, whereas the complete predicted ordering was not established for either control measure.

H2a was supported under the primary Bayesian specification, with reported detection estimated as higher in high- than in low-criticality tasks. This conclusion was sensitive to task-pair adjustment and to the treatment of indeterminate reports. H2b and H2c were not supported, and the descriptive H2d expectation about detection stage was not supported. The RQ2 conclusions concern assigned-error trials classified as exposed and determinate; their condition and condition-by-criticality comparisons have limited precision after these eligibility and classification constraints.

5.5 Secondary and Exploratory Outcomes

The remaining outcomes provide descriptive context for RQ1 and RQ2. They cover perceived results, task duration, recorded corrections, additional questionnaire items, oversight preferences, and written interview themes.

5.5.1 Perceived Result Match and Assessment Confidence

Result-match and assessment confidence ratings describe the perceived outcomes.

Across the 108 tasks, participants rated the result as completely matching the intended task outcome in 78 cases (72.2%). This included all 54 trials without an assigned controlled error and 24 of the 54 trials with an assigned controlled error (44.4%). Table 5.8 shows the complete response distribution and the associated confidence ratings.

Table 5.8: Perceived result match and assessment confidence for tasks with and without an assigned error. Result-match values are counts.

Result-match response	Assigned error	No assigned error	Total
Not at all	4	0	4
Rather not	6	0	6
Partly	19	0	19
Largely	1	0	1
Completely	24	54	78
Cannot assess	0	0	0
Confidence, median [IQR]	7 [6, 7]	7 [6, 7]	7 [6, 7]

No participant selected the option that the result could not be assessed. Complete-match ratings were slightly more frequent in C1, while median assessment confidence was 7 [6, 7] in every condition. These ratings describe perceived result quality. A controlled error could have been corrected before the final result, so a complete-match rating does not by itself indicate that the error was missed.

5.5.2 Trial Duration

Total task duration is the sum of the recorded execution intervals for each of the 108 scheduled tasks. Table 5.9 reports the distributions by oversight condition.

Table 5.9: Total duration of all scheduled tasks by oversight condition. Durations are in seconds and reported as medians with interquartile ranges in brackets.

Condition	<i>n</i>	Duration
C1	36	47.52 [36.80, 58.96]
C2	36	33.78 [27.11, 44.22]
C3	36	25.99 [23.47, 41.05]

C1 had the longest observed median duration and C3 the shortest. These durations include agent execution and participant waiting. They exclude apparatus-reset time and are not interpreted as a direct measure of participant effort.

5.5.3 Recorded Explicit Corrections

The runtime audit identifies accepted explicit-correction chains in assigned-error tasks. A chain was counted when the audit recorded both acceptance of a participant correction and the resulting rewind of the active trial.

The runtime audit contained 17 accepted explicit-correction chains across 16 of the 54 tasks with an assigned controlled error (29.6%). Fifteen tasks contained one such chain, one task contained a chain in each of two technical attempts, and 38 contained none. Table 5.10 separates the counts by oversight condition and task criticality.

Table 5.10: Recorded explicit corrections among tasks with an assigned error. Values are counts.

Group	Error tasks	Tasks with correction	Correction chains
C1	18	7	8
C2	18	5	5
C3	18	4	4
Low criticality	27	4	5
High criticality	27	12	12

Recorded corrections were more frequent in C1 and in high-criticality tasks (Table 5.10). These descriptive counts cover persisted correction events, not all detection or intervention. The absence of a recorded correction does not mean that the error went unnoticed, because a participant could report it without correcting it. The retained audit exports contain no dedicated participant pause or stop events, so their frequencies cannot be determined.

5.5.4 Confirmation Frequency and Attention Cues

Figure 5.3 shows the complete response distributions for the two exploratory control-context items. Confirmation- frequency ratings were divided in C1, high in C2, and clustered at 2 and 4 in C3.

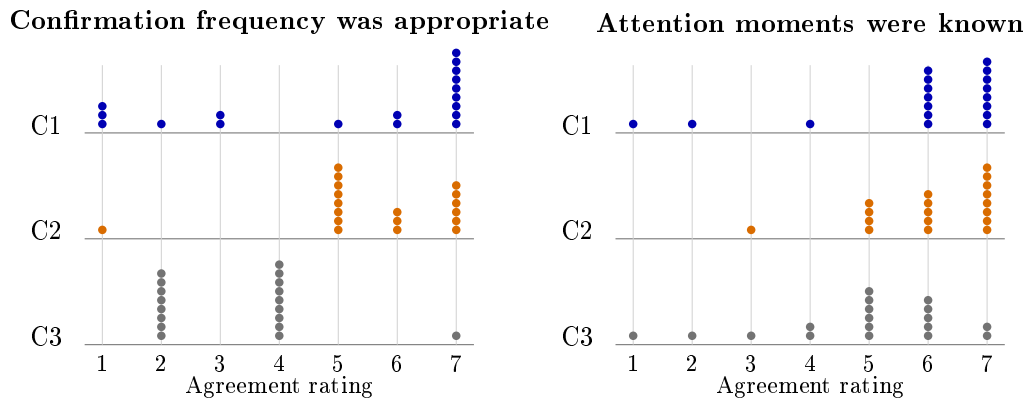


Figure 5.3: Exploratory control-context ratings by oversight condition. Each dot represents one participant response on the seven-point agreement scale from 1 (strongly disagree) to 7 (strongly agree), with $n = 18$ per condition.

Ratings of 6 or 7 for knowing when particular attention was required declined from 15 of 18 participants in C1 to 13 in C2 and 7 in C3. These exploratory distributions are descriptive only.

5.5.5 Understanding, Predictability, and Trust

Four adapted items assessed understanding and predictability, while two assessed trust. The understanding and predictability items received high ratings in all conditions (Table 5.11). For understanding why Caddie executed an action, 17 of 18 participants selected 6 or 7 in each condition.

The trust items received lower ratings than the understanding and predictability items. Median trust in Caddie's decisions was descriptively highest in C2 and lowest in C3. The six adapted items remain separate exploratory measures, and no condition effect is tested.

Table 5.11: Individual exploratory understanding, predictability, and trust ratings by oversight condition. Values are medians with interquartile ranges in brackets on the seven-point agreement scale.

Item	C1	C2	C3
Understood why an action was executed	7 [7, 7]	7 [7, 7]	7 [6.25, 7]
Could predict the next action	7 [6, 7]	7 [7, 7]	6 [6, 7]
Behavior was understandable from what was visible	7 [6, 7]	7 [6, 7]	6 [6, 7]
Displayed actions were predictable	7 [6.25, 7]	7 [7, 7]	7 [7, 7]
Trusted Caddie to make the right decisions	4 [3, 4.75]	5 [4, 5]	3 [2, 4]
Comfortable leaving decisions to Caddie	4 [3, 4]	4 [4, 5]	3 [3, 4.75]

5.5.6 Overall and Consequence-Specific Oversight Preferences

An overall ranking and separate choices for low- and high-consequence tasks describe the participants' oversight preferences.

C2 was ranked first overall by 10 of 18 participants, followed by C1 for six participants and C3 for two. C2 had mean rank 1.50, compared with 2.11 for C1 and 2.39 for C3, where rank 1 denotes the highest preference. Figure 5.4 shows participant-level transitions between preferred conditions for low- and high-consequence tasks.

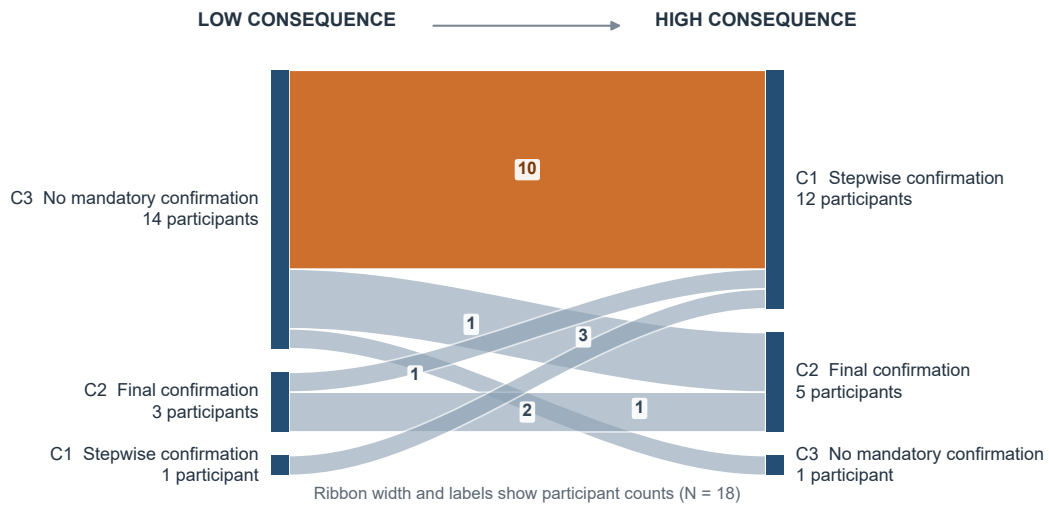


Figure 5.4: Transitions between preferred oversight conditions for low- and high-consequence tasks. Ribbon width and labels represent participant counts ($N = 18$).

Among the 14 participants who selected C3 for low-consequence tasks, ten selected C1 for high-consequence tasks, three selected C2, and one retained C3. Of the other four, one retained C1, two retained C2, and one changed from C2 to C1. These preference responses are reported descriptively without an inferential test.

5.5.7 Interview Themes

The written interview responses describe usefulness, control, attention, and delegation boundaries. Across the 108 responses, 104 received at least one specific code, meaning a thematic label for the response’s content. Four short responses did not meet a specific code definition, and none was marked unresolved. Twelve participants described Caddie as practical or useful. Table 5.12 summarizes the main oversight, attention, delegation, and error-related themes. Counts refer to distinct participants who expressed a theme in at least one of their six responses. The same participant can appear in several rows.

One paraphrased coding example illustrates this overlap: a response valued C2 for combining speed with control and described stepwise confirmation as burdensome. It received both the C2-checkpoint and C1-confirmation-burden codes.

Table 5.12: Main themes in participants’ written interview responses. Values are deduplicated counts among the 18 participants.

Coded theme	Participants
C1 provided control or safety	10
C1 confirmations created burden	11
C2 was a balanced or useful final checkpoint	12
C3 reduced control or increased perceived risk	7
C3 control depended on participant attention	5
Financial or numerical content received closer attention	12
Attention was allocated by possible consequences	11
Attention was relaxed for lower-risk tasks	9
Participants wanted approval for financial actions	16
Participants wanted approval for important time commitments	11
Oversight should adapt to the task	11
At least one anomaly was recalled	18
Multiple anomalies were recalled	11
An error was seen late or not stopped	7
An error correction was recalled	4

Participants associated C1 with greater control or safety, but also with repeated-confirmation burden. C2 was most often described as a balanced or useful checkpoint. C3 was linked to reduced control or increased perceived risk and to control that depended on active attention.

Participants most often wanted approval for financial actions, followed by important time commitments. All 18 participants recalled at least one anomaly, but only four mentioned a correction. These interview counts remain separate from the trial-level RQ2 classifications. They describe the retained written responses and are not inferential estimates of population prevalence.

6 Discussion

This chapter interprets the study findings as evidence about trade-offs in required oversight rather than as a ranking of one universally best condition. It first considers interaction effort, workload, and control, then relates participant-reported error detection to task criticality before deriving design implications and limitations. The interpretation is compared with the closest empirical work from Chapter 2.

6.1 Effects of Required Oversight

The first research question examined how the three oversight conditions affected required interaction, workload, perceived control, and timely intervention. These outcomes are considered together because additional confirmations can create both observable interaction costs and opportunities for control.

The H1a ordering reflects the interaction cost built into the oversight conditions: more required decisions also meant more time spent responding to confirmations (Table 5.2). These measures describe the participant’s required interaction, not a separate performance advantage. The corresponding task-duration pattern provides context but is not a pure measure of participant effort because it also includes execution (Table 5.9). Neither pattern shows that the underlying agent was less efficient in C1.

C2 had the lowest descriptive workload even though C3 required no decisions (Table 5.3). The observed workload pattern therefore did not mirror the number of required decisions. Fewer button presses need not mean less mental demand, but the planned comparisons did not support H1b and did not establish a workload advantage for C2.

In a simulated daily-assistant experiment, He et al. found that involving users in both planning and execution increased cognitive load, although involvement could also help users repair imperfect plans or actions [17]. Caddie manipulated required oversight during execution rather than planning involvement. The comparison therefore concerns the interaction cost of user involvement, not a replication of their workload effect.

The control findings similarly point to the placement of an intervention opportunity rather than only its frequency. C2 received higher timely-intervention ratings than C3, while C1 did not show a clear advantage over C2 (Table 5.4). A checkpoint immediately before the consequential commit may therefore have made the opportunity to intervene sufficiently clear. The C2–C3 difference was supported by the ordinal model but not by the Holm-adjusted paired-rank sensitivity. H1c therefore received

partial, method-dependent support, concerning experienced control rather than actual intervention performance.

The overall preference for C2 and 12 descriptions of it as a balanced or useful checkpoint support interpreting it as a compromise rather than a uniformly superior condition. These preferences came from the same participants as the workload and control ratings, so they do not independently confirm a causal advantage.

A comparison of model-selected checkpoints by Zhou et al. similarly shows why confirmation placement should not be reduced to the number of checks. Their intermediate checkpoints in simulated desktop tasks were associated with shorter mean completion times than end confirmation and were generally preferred [42]. Caddie’s C1 instead required confirmation before each configured task-advancing step. The comparison points to checkpoint placement as well as frequency, without establishing C2 as generally optimal.

The consequence-specific preferences further argue against one fixed oversight level for every task. Fourteen participants preferred C3 for low-consequence tasks, whereas 12 preferred C1 for high-consequence tasks (Figure 5.4). Eleven participants also described a need to adapt oversight to the task. The pattern suggests that the acceptable interaction cost depends on what could happen if the agent acts incorrectly. Because these measures were exploratory, they support a task-dependent account of preferred oversight but do not establish which condition produces the best objective outcome for a given level of consequence.

The absence of one uniformly preferred condition is also consistent with a live-web comparison of four broader oversight strategies by Chen et al. No strategy was consistently best across workload, perceived control, trust, and usability. The clearest trust pattern depended on task context [9]. The four conditions combined several planning, execution, and interface mechanisms, whereas Caddie varied the structure of required confirmation during controlled smartphone tasks. The similarity supports a context-dependent account of oversight, not a shared causal mechanism or a validated policy threshold.

6.2 Participant-Reported Detection of Introduced Errors Across Tasks

The second research question examined how often participants retrospectively reported detecting the introduced error in low- and high-criticality tasks and across oversight conditions C1, C2, and C3. The discussion first considers the difference between the two task sets and then examines the pattern across the three conditions.

Participants were more likely to report detecting introduced errors in the selected high-criticality tasks. They also rated these tasks as more consequential, and recorded corrections were descriptively more common in them (Table 5.10). However, each task appeared at only one criticality level and differed in its content and introduced error.

The study therefore cannot show that perceived consequence itself caused the detection difference. The primary analysis supports H2a for the selected tasks. The sensitivity checks still estimated higher detection for high-criticality tasks. However, the 95% credible intervals also included no difference when an alternative model was used or when the 11 unclear detection reports were treated as non-detections (Section 5.4.1).

No reliable difference in reported detection among oversight conditions was established, and H2b was not supported (Table 5.6). The estimated high-minus-low differences were positive in all three conditions, but the data did not establish their predicted variation across conditions (H2c).

One possible explanation is that required confirmation and active attention are not the same. Repeated C1 prompts may have encouraged routine approval, while participants in C3 may have monitored the visible execution more continuously. The 11 participants who described C1 confirmation burden and the polarized confirmation-frequency ratings provide context for this interpretation (Table 5.12 and Figure 5.3). Participants may also have checked more carefully when an error seemed more consequential. However, recorded corrections do not directly measure attention, and the study did not record whether prompts were read or how attention changed over time. These remain possible explanations, not measured causes of the detection pattern.

A related distinction appears in retrospective agent-review work by Grunde-McLaughlin et al., which found a small estimated reduction in verification time without an established accuracy improvement. Confidence could increase when participants incorrectly accepted a faulty result [16]. Their Wizard-of-Oz review setting concerned completed question-answering runs, whereas Caddie examined retrospective reports after live smartphone execution. These differences limit a direct comparison of the two review settings.

H2d was not supported, and the reported stages further separate mandatory checkpoints from when participants said they noticed an error. Most detections in C1 and C3 were reported during execution, whereas C2 contained more result-only reports. The timing pattern therefore did not group C1 and C2 together as predicted. Because participants reported the stage retrospectively and the questionnaire could not identify a particular gate or checkpoint, these data describe perceived timing rather than the exact moment of recognition (Table 5.7).

6.3 Design Implications for Shared Autonomy

The findings motivate adapting review to the task and making consequential actions easy to inspect.

Participants' preferences suggest adapting confirmation to the consequences of an action (Section 6.1). Lower-consequence actions could proceed with fewer mandatory confirmations, while actions with greater potential consequences would receive explicit review. The study did not establish an optimal confirmation policy or validate a threshold for deciding when review is needed.

The review itself should highlight the task parameters that determine an action’s consequences, such as the recipient, amount, destination, or scheduled time, while the action can still be changed. The control findings in Section 6.1 motivate making the opportunity to intervene easy to identify. However, the detection results in Section 6.2 do not establish that one final checkpoint is sufficient to ensure error recognition. Clearer review information remains a design recommendation to evaluate, not a demonstrated improvement in detection.

This design direction is compatible with the selective policy already implemented in Caddie’s normal mode, but the study did not test that policy. The Android-owned loop lets ordinary actions proceed automatically. When the local classifier identifies a proposed action as consequential, the loop pauses and waits for the user’s decision. The action is dispatched only after approval. The experiment instead used fixed C1–C3 gates. Its findings are consistent with consequence-dependent confirmation, but they neither validate the classifier’s rules nor show that it identifies every consequential action.

6.4 Limitations

The design supports comparisons among the three conditions for the selected tasks, participant sample, and measurement procedure. These boundaries limit generalizability and causal interpretation.

The analysis cohort of 18 participants covered all planned matrix assignments, but its small size makes estimates imprecise, especially for the Oversight $\times$ Criticality interaction after exposure and determinacy exclusions and for the detection-stage summary based on 35 detected trials. Its size was selected for design balance rather than an a priori power calculation. A non-significant condition difference is not evidence of equivalence. The convenience sample came from personal, workplace, and private social networks, and all participants were aged 22–35 years. Findings may not generalize beyond these groups, and personal relationships may have affected participation or socially desirable responses.

All three conditions were completed in one session, with two consecutive tasks assigned to each condition. The design matrix balanced conditions across trial periods, and period was included in the analyses. These measures reduce order-related bias but cannot rule out residual practice, fatigue, or carryover effects. Each participant encountered every task only once, so the findings describe first-session experience rather than how oversight behavior might change with repeated everyday use.

Each concrete task and introduced error appeared at only one predefined criticality level. Participants rated the selected high-criticality tasks as more consequential, but criticality remained tied to task content, application, and how visible the error was. In Maps/Messenger, for example, the incorrect destination Cologne Central Station may have appeared only briefly. Brief visibility is also plausible in everyday agent use, but its duration was not varied independently. The observed difference between high- and

low-criticality tasks therefore cannot be attributed to criticality alone. Participants reported detection and its stage after each task to avoid interrupting execution or directing their attention toward errors. This reduces reactivity, but these retrospective reports cannot identify the exact moment of recognition, the relevant C1 gate or C2 checkpoint, or whether the participant inspected the necessary screen content.

The primary RQ2 outcome also depends on classifying written anomaly reports and technical exposure evidence. The final classifications of all 54 assigned-error trials were reviewed and confirmed against the predefined rules. This was a single-researcher assessment, not an independent evaluation by two human coders. This assessment cannot establish independent agreement or eliminate possible interpretation bias.

The RQ1 timing measure captures responses to mandatory confirmations, not overall attention or monitoring. C3 therefore contributes zero confirmation-response time by design even when a participant watches progress. The P02–P19 values are response latencies to required C1 and C2 confirmation points recorded during trial execution.

Perceived control and timely intervention were measured as participant ratings, not as comprehensive records of actual intervention performance. The retained audit exports contain explicit correction events but no dedicated pause or stop events, so voluntary intervention frequency cannot be reconstructed. The study also retained no independent physical-touch validation of the C3 pause path. Consequently, it cannot establish whether the conditions changed the frequency or technical success of voluntary intervention.

The trials used deterministic task specifications, seven resettable local fixture applications, and Google Maps as a live external surface. Fixtures gave repeatable starting states and avoided personal accounts or real financial transactions, but the errors could not have the full real-world consequences represented by the scenarios. Fixed C1–C3 gates did not evaluate open-ended model action selection or the normal-mode classifier. The findings therefore concern oversight in controlled tasks, not normal-mode reliability or effectiveness. Development trials with off-phone models recorded incorrect actions and false completion claims (Section 3.7), but do not establish the current normal-mode success rate.

On-device generative execution remains untested. Caddie’s Android runtime controls execution, but the language model runs on a separate computer (Section 3.8). A practical configuration would need reliable task performance and acceptable device resource use and response times, which were not evaluated together on a smartphone.

Self-hosting also does not mean that all information stays on the phone. In normal mode, task descriptions and observed application content can be sent to the separate model computer. Filtering text from password fields does not remove sensitive text displayed elsewhere (Section 3.8). The model request precedes approval of the proposed action, so relevant context may already have left the phone when a confirmation is shown. The checkpoint controls action execution, not the preceding transmission of information used to select it.

7 Conclusion and Future Work

The conclusions bring together Caddie’s technical validation and the study findings on required oversight and reported error detection. They also identify questions for further development and evaluation beyond the present study.

7.1 Conclusion

Two forms of evidence support the conclusion: technical validation of Caddie’s oversight mechanisms and empirical comparisons of C1, C2, and C3.

Caddie combines cross-application action execution with a persistent view of the current state and action, together with controls for confirmation, pausing, correction, and stopping. Its model-driven normal mode and deterministic study mode are separate execution paths that reuse the same participant-facing oversight interface. This separation enabled C1, C2, and C3 to vary required confirmation while action visibility and the opportunity to intervene remained available. The technical evidence supports these oversight mechanisms in the documented test configurations. It does not establish that Caddie’s model can reliably complete arbitrary smartphone tasks.

For RQ1, greater required oversight was associated with more required decisions and longer cumulative response time. This effort measure partly reflects the condition design, while subjective outcomes did not follow a simple more-oversight-is-better pattern. C2 required fewer decisions than C1, had the descriptively lowest workload, and received higher timely-intervention ratings than C3. The planned workload contrasts did not support H1b, and H1c was only partially supported, with the timely-intervention result depending on the analysis method. Required confirmation shaped interaction cost but did not by itself determine workload or experienced control.

For RQ2, the primary analysis found that participants were more likely to report detecting introduced errors in the selected high-criticality tasks than in the selected low-criticality tasks. This supports the predicted direction of H2a for these specific tasks, although support depended on the model and the handling of unclear reports. No reliable difference in reported detection among oversight conditions was established, and H2b was not supported. H2c and descriptive H2d were also not supported. Required decision points therefore did not guarantee reported detection. The study also cannot determine whether the difference between criticality levels resulted from task consequences themselves, task content, error visibility, or retrospective reporting.

Across both research questions, the findings do not identify one universally best oversight condition. Participants’ preferences changed with the potential consequences of the task. The findings therefore support clear review opportunities before consequential actions while suggesting that ordinary actions may proceed with less mandatory interruption. The study does not define a general threshold for consequential actions or validate a particular adaptive classifier.

7.2 Future Work

Future work should extend both Caddie’s normal mode and the evidence for how its oversight behaves outside controlled deterministic trials. The most relevant directions concern adaptive checkpoints, alternative execution interfaces, verifiable outcomes, and repeated use.

The first priority is to evaluate how Caddie’s normal-mode consequence classifier decides which actions require approval, rather than treating a fixed C1–C3 condition as a deployment policy. A shadow-mode test could record its decisions without executing actions, then compare them with predefined criteria and users’ judgments. A later study could compare a fixed final checkpoint with an adaptive policy and editable rules for particular action types. Each checkpoint should show the pending state change and its recipient, value, destination, or scheduled time while correction remains possible. The evaluation should distinguish allowing a consequential action without confirmation from unnecessarily interrupting a routine one. Workload, response time, and successful correction could measure checkpoint burden and whether it helped prevent errors. This would test an adaptive smartphone policy in light of prior work on context-dependent oversight and checkpoint scheduling [9, 42].

A second direction is to compare graphical execution with a hybrid path for supported workflows while keeping checkpoint information, logs, and intervention controls unchanged. The hybrid path could use Android AppFunctions for local structured actions, Accessibility for fallback and visual verification, and MCP for optional remote services. AppFunctions remains experimental, and cross-application execution depends on platform access checks [4, 5]. Relevant outcomes include task success, fragile interface steps, parameter errors, and users’ ability to inspect the pending action. MobileWorld demonstrates an MCP-to-graphical-action task pattern, but does not compare such paths under a shared human-oversight policy [20].

Third, future work should strengthen the checks that determine whether an action actually succeeded. Interface-changing normal-mode actions already state the expected result after execution. Application-specific checks could classify the outcome as verified success, verified failure, or still unclear. Caddie should repeat an action automatically only when repeating it cannot create an additional unwanted effect. It should offer a recovery action only when that action’s effect is known. VeriGUI similarly makes action-effect verification and recovery explicit parts of a GUI-agent loop [41]. Applying this principle to Caddie would require evidence collected on Android rather than relying only on the acting model’s judgment. For on-device processing, nearer-term

7 Conclusion and Future Work

targets are policy enforcement, privacy filtering, and deterministic verification. Full on-device generative execution should be considered only after adequate normal-mode task performance and device testing of memory use, latency, energy consumption, and thermal behavior.

Finally, these extensions should be evaluated in the normal mode that they are intended to improve. The first step is a reproducible task benchmark with independently checked end states, varied starting states, and injected network, rendering, and application interruptions. It should report task success, false success claims, duplicate side effects, pauses for human review, and the number and duration of model and tool calls. A subsequent multi-day study could test whether people continue to inspect checkpoints after becoming familiar with the interface. It should vary three factors separately: how consequential an action is, whether it can be reversed, and how long an incorrect value remains visible. The system should record when an action is proposed, shown for review, dispatched, and interrupted. This design would distinguish routine approval from task learning and would address the present study’s one-session design, retrospective detection reports, and task-specific criticality comparison.

Bibliography

- [1] David A. Abbink et al. “A Topology of Shared Control Systems—Finding Common Ground in Diversity”. In: *IEEE Transactions on Human-Machine Systems* 48.5 (2018), pp. 509–525. DOI: 10.1109/THMS.2018.2791570.
- [2] Android Developers. *Android Debug Bridge (adb)*. 2026. URL: <https://developer.android.com/tools/adb> (visited on 07/28/2026).
- [3] Android Developers. *Create an Accessibility Service*. 2026. URL: <https://developer.android.com/guide/topics/ui/accessibility/service> (visited on 07/27/2026).
- [4] Android Developers. *Manifest.permission.EXECUTE_APP_FUNCTIONS*. 2026. URL: https://developer.android.com/reference/android/Manifest.permission#EXECUTE_APP_FUNCTIONS (visited on 08/18/2026).
- [5] Android Developers. *Overview of AppFunctions*. 2026. URL: <https://developer.android.com/ai/appfunctions> (visited on 08/18/2026).
- [6] Android Developers. *UiDevice API Reference*. 2026. URL: <https://developer.android.com/reference/androidx/test/uiautomator/UiDevice> (visited on 08/09/2026).
- [7] Apple. *Apple Introduces Siri AI, a Profoundly More Capable and Personal Assistant*. June 8, 2026. URL: <https://www.apple.com/newsroom/2026/06/apple-introduces-siri-ai-a-profoundly-more-capable-and-personal-assistant/> (visited on 08/17/2026).
- [8] Yiheng Bian et al. “DroidRetriever: A Transparent and Steerable Automation System for Collaborative Mobile Information Seeking”. In: *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. CHI ’26. Association for Computing Machinery, Apr. 2026, pp. 1–24. DOI: 10.1145/3772318.3790396.
- [9] Chaoran Chen et al. *Comparing Human Oversight Strategies for Computer-Use Agents*. 2026. DOI: 10.48550/arXiv.2604.04918. arXiv: 2604.04918 [cs.HC]. URL: <https://arxiv.org/abs/2604.04918>.
- [10] Aakanksha Chowdhery et al. “PaLM: Scaling Language Modeling with Pathways”. In: *Journal of Machine Learning Research* 24.240 (2023), pp. 1–113. URL: <https://jmlr.org/papers/v24/22-1144.html>.
- [11] Rune Haubo B. Christensen. *Cumulative Link Models for Ordinal Regression with the R Package ordinal*. Package vignette; undated version. URL: https://cran.r-project.org/web/packages/ordinal/vignettes/clm_article.pdf (visited on 09/02/2026).

Bibliography

- [12] Mica R. Endsley and Esin O. Kiris. “The Out-of-the-Loop Performance Problem and Level of Control in Automation”. In: *Human Factors* 37.2 (1995), pp. 381–394. DOI: 10.1518/001872095779064555.
- [13] Milton Friedman. “The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance”. In: *Journal of the American Statistical Association* 32.200 (1937), pp. 675–701. DOI: 10.1080/01621459.1937.10503522.
- [14] Google. *Let Gemini Handle Your Multi-Step Daily Tasks on Android*. Feb. 25, 2026. URL: <https://blog.google/innovation-and-ai/products/gemini-app/android-multi-step-tasks/> (visited on 08/17/2026).
- [15] Sander Greenland et al. “Statistical Tests, *P* Values, Confidence Intervals, and Power: A Guide to Misinterpretations”. In: *European Journal of Epidemiology* 31.4 (2016), pp. 337–350. DOI: 10.1007/s10654-016-0149-3.
- [16] Madeleine Grunde-McLaughlin et al. *Overseeing Agents Without Constant Oversight: Challenges and Opportunities*. 2026. DOI: 10.48550/arXiv.2602.16844. arXiv: 2602.16844 [cs.HC]. URL: <https://arxiv.org/abs/2602.16844>.
- [17] Gaole He, Gianluca Demartini, and Ujwal Gadiraju. “Plan-Then-Execute: An Empirical Study of User Trust and Team Performance When Using LLM Agents As A Daily Assistant”. In: *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2025, pp. 1–22. DOI: 10.1145/3706598.3713218.
- [18] Sture Holm. “A Simple Sequentially Rejective Multiple Test Procedure”. In: *Scandinavian Journal of Statistics* 6.2 (1979), pp. 65–70. URL: <https://www.jstor.org/stable/4615733>.
- [19] Jeonghyeon Kim et al. *AgentLens: Adaptive Visual Modalities for Human-Agent Interaction in Mobile GUI Agents*. 2026. arXiv: 2604.20279 [cs.HC]. URL: <https://arxiv.org/abs/2604.20279>.
- [20] Quyu Kong et al. “MobileWorld: Benchmarking Autonomous Mobile Agents in Agent-User Interactive and MCP-Augmented Environments”. In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 6142–6167. DOI: 10.18653/v1/2026.acl-long.278. URL: <https://aclanthology.org/2026.acl-long.278/>.
- [21] Moritz Körber. “Theoretical Considerations and Development of a Questionnaire to Measure Trust in Automation”. In: *Proceedings of the 20th Congress of the International Ergonomics Association (IEA 2018): Volume VI: Transport Ergonomics and Human Factors (TEHF), Aerospace Human Factors and Ergonomics*. Ed. by Sebastiano Bagnara et al. Springer International Publishing, 2019, pp. 13–30. DOI: 10.1007/978-3-319-96074-6_2. URL: https://doi.org/10.1007/978-3-319-96074-6_2.

Bibliography

- [22] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [23] Juha Matero and Ashley Colley. “Identifying Unintentional Touches on Hand-held Touch Screen Devices”. In: *Proceedings of the Designing Interactive Systems Conference*. DIS '12. New York, NY, USA: Association for Computing Machinery, 2012, pp. 506–509. DOI: 10.1145/2317956.2318031. URL: <https://doi.org/10.1145/2317956.2318031>.
- [24] Model Context Protocol. *Tools*. Nov. 25, 2025. URL: <https://modelcontextprotocol.io/specification/2025-11-25/server/tools> (visited on 08/09/2026).
- [25] National Aeronautics and Space Administration. *NASA Task Load Index (TLX)*. Mar. 11, 2026. URL: <https://www.nasa.gov/human-systems-integration-division/nasa-task-load-index-tlx/> (visited on 08/18/2026).
- [26] National Institute of Standards and Technology. *Median Confidence Limits*. NIST Dataplot Reference Manual. URL: <https://www.itl.nist.gov/div898/software/dataplot/refman1/auxillar/mediancl.htm> (visited on 09/03/2026).
- [27] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. “A Model for Types and Levels of Human Interaction with Automation”. In: *IEEE Transactions on Systems, Man, and Cybernetics–Part A: Systems and Humans* 30.3 (2000), pp. 286–297. DOI: 10.1109/3468.844354.
- [28] PyMC Developers. *pymc.sample*. PyMC 6.3.1 API documentation. URL: <https://www.pymc.io/projects/docs/en/stable/api/generated/pymc.sample.html> (visited on 09/02/2026).
- [29] Christopher Rawles et al. “AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents”. In: *International Conference on Learning Representations*. 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/01a83bc2f2732a58e6aa731e659e7101-Abstract-Conference.html.
- [30] Samsung. *How to Use Seamless Actions across Apps on Galaxy Devices*. URL: <https://www.samsung.com/us/support/answer/ANS10010644/> (visited on 08/17/2026).
- [31] Timo Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: https://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- [32] S. S. Shapiro and M. B. Wilk. “An Analysis of Variance Test for Normality (Complete Samples)”. In: *Biometrika* 52.3–4 (1965), pp. 591–611. DOI: 10.1093/biomet/52.3-4.591.
- [33] Ben Shneiderman. “Human-Centered Artificial Intelligence: Reliable, Safe & Trustworthy”. In: *International Journal of Human–Computer Interaction* 36.6 (2020), pp. 495–504. DOI: 10.1080/10447318.2020.1741118.

Bibliography

- [34] Stan Development Team. *How to Diagnose and Resolve Convergence Problems*. URL: <https://mc-stan.org/learn-stan/diagnostics-warnings.html> (visited on 09/02/2026).
- [35] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [36] Ronald L. Wasserstein and Nicole A. Lazar. “The ASA’s Statement on p -Values: Context, Process, and Purpose”. In: *The American Statistician* 70.2 (2016), pp. 129–133. DOI: 10.1080/00031305.2016.1154108.
- [37] Hao Wen et al. “AutoDroid: LLM-Powered Task Automation in Android”. In: *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 2024, pp. 543–557. DOI: 10.1145/3636534.3649379. URL: <https://doi.org/10.1145/3636534.3649379>.
- [38] Frank Wilcoxon. “Individual Comparisons by Ranking Methods”. In: *Biometrics Bulletin* 1.6 (1945), pp. 80–83. DOI: 10.2307/3001968.
- [39] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations*. 2023. URL: <https://arxiv.org/abs/2210.03629>.
- [40] Chi Zhang et al. “AppAgent: Multimodal Agents as Smartphone Users”. In: *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2025. DOI: 10.1145/3706598.3713600.
- [41] Yuzhe Zhang et al. “Don’t Act Blindly: Robust GUI Automation via Action-Effect Verification and Self-Correction”. In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 28924–28941. DOI: 10.18653/v1/2026.acl-long.1335. URL: <https://aclanthology.org/2026.acl-long.1335/>.
- [42] Jieyu Zhou et al. “When Should Users Check? Modeling Confirmation Frequency in Multi-Step Agentic AI Tasks”. In: *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. CHI ’26. Association for Computing Machinery, Apr. 2026, pp. 1–20. DOI: 10.1145/3772318.3790655.

Declaration of Independent Authorship

I declare that I prepared this thesis independently. I have identified all passages derived directly or indirectly from published or unpublished work and have listed every source and aid used in preparing the thesis.

This thesis, in the same or substantially similar form, has not previously been submitted to another examination authority.

Place and date

Signature

Use of Generative AI

Generative AI formed part of the technical subject and research procedure. It also supported selected steps in software development, analysis, visualization, writing, and review. The tools and their uses are summarized below.

Tool or system	Purpose	Use and verification
OpenAI Codex and Anthropic Claude Code	Development and documentation	Supported coding, debugging, refactoring, and documentation. Suggestions were reviewed against the implementation, and relevant builds or tests were run before technical claims were retained.
OpenAI Codex	Analysis, visualization, writing, and review	Supported analysis scripts, visual alternatives, English-language revision, consistency checks, and critical reviews. Numerical results, sources, and final wording were checked against retained data and original sources.
qwen3.6-35b-a3b and Claude	Research systems	The Qwen model was used in Caddie's normal-mode evaluation. Claude, recorded by the runner as Opus 4.7, was used in a separate reference run during development. Outputs were treated as experimental observations, not scholarly sources.
OpenAI image generation	Visual exploration	Used to explore possible figure directions. No AI-generated raster illustration was retained in the submitted thesis.

Generative AI was most useful for repetitive implementation work, exploring alternatives, language feedback, and systematic consistency checks. It was less reliable for exact source support, methodological interpretation, and final visual layout. Suggestions could be incomplete, overly generic, or incorrect and were therefore revised or rejected after comparison with original sources, code, tests, and retained study data. I remain responsible for the research decisions, analysis, argument, and final submitted text.